

OAuth 2.0

Guida Completa — Dalla Storia ai Dettagli Implementativi

Storia • Protocolli • JWT • PKCE • OpenID Connect • Python • IoT/PLC

Argomento	OAuth 2.0 + OpenID Connect
Esempi	Python 3.10+
Livello	Base → Avanzato
Capitoli	12 capitoli + appendice

Sommario

Capitolo 1: L'Era Pre-OAuth — Il Problema delle Password	8
1.1 Il Web 2.0 e l'Esplosione delle API	8
1.2 L'Anti-Pattern della Password Sharing	8
1.3 I Tentativi Pre-OAuth e il Fallimento della Frammentazione	8
Capitolo 2: Nascita di OAuth 1.0 (2007–2010)	10
2.1 OAuth 1.0: Il Flusso Three-Legged	10
2.2 Il Problema della Firma e Perché Serve OAuth 2.0	10
Capitolo 3: OAuth 2.0 — Framework, Attori e Token	12
3.1 I Quattro Attori	12
3.2 Access Token, Refresh Token e Authorization Code	12
3.3 Gli Scope — Granularità dei Permessi	12
Capitolo 4: Authorization Code Flow	14
4.1 Registrazione dell'App	14
4.2 Il Flusso Passo per Passo	14
4.3 Risposta Token — Analisi Completa	15
Capitolo 5: PKCE — Proof Key for Code Exchange	16
5.1 Il Problema: Authorization Code Interception	16
5.2 Meccanismo PKCE — Code Verifier e Code Challenge	16
5.3 Perché PKCE è Ora per Tutti	17
Capitolo 6: Gli Altri Grant Type	18
6.1 Client Credentials — Machine-to-Machine	18
6.2 Device Authorization Grant (Device Flow)	19
6.3 Implicit e ROPC — Flussi Deprecati	19

Capitolo 7: JWT — JSON Web Token	20
7.1 Struttura: header.payload.signature	20
7.2 Algoritmi di Firma	20
7.3 Validazione Corretta di un JWT	20
Capitolo 8: OpenID Connect (OIDC)	22
8.1 OAuth 2.0 vs OpenID Connect	22
8.2 Discovery Endpoint e Auto-Configurazione	22
8.3 Login con Google — Implementazione Flask Completa	22
Capitolo 9: Refresh Token, Revoca e Ciclo di Vita	24
9.1 Refresh Token Flow	24
9.2 Token Introspection — RFC 7662	25
9.3 Best Practice per lo Storage dei Token	25
Capitolo 10: Sicurezza OAuth 2.0 — Attacchi e Difese	26
10.1 Tabella degli Attacchi Principali	26
10.2 JWT Algorithm Confusion — Dettaglio	26
10.3 Checklist di Sicurezza OAuth 2.0	26
Capitolo 11: OAuth 2.0 nell'Automazione Industriale	28
11.1 Sfide dell'Automazione Industriale	28
11.2 Pattern: Edge OAuth Proxy per PLC	28
11.3 Esempio: Impianto Trattamento Acque	29
Capitolo 12: Provider Reali e Librerie Python	31
12.1 Google — google-auth e Workspace API	31
12.2 Microsoft Azure AD con MSAL	31
12.3 GitHub OAuth — requests-oauthlib	32
12.4 FastAPI Resource Server — Protezione Endpoint	32

Capitolo 13: HTTP e TLS — Fondamenta di OAuth 2.0	34
13.1 Anatomia di una Richiesta OAuth	34
13.2 HTTP Status Code in OAuth	34
13.3 TLS — Il Prerequisito Assoluto	35
Capitolo 14: Gestione degli Errori OAuth 2.0	36
14.1 Errori del Token Endpoint	36
14.2 Client OAuth Robusto con Retry	36
14.3 Errori del Authorization Endpoint	37
Capitolo 15: Testing di Applicazioni OAuth	38
15.1 Mock dell'Authorization Server	38
15.2 Integration Testing con unittest.mock	39
Capitolo 16: Performance e Caching OAuth	41
16.1 Caching delle Chiavi JWKS	41
16.2 Benchmark: Con e Senza Cache	42
16.3 Token Caching con Redis	42
Capitolo 17: Single Sign-On con OpenID Connect	44
17.1 Come Funziona il SSO con OIDC	44
17.2 SSO Logout — Front-Channel e Back-Channel	44
Capitolo 18: Keycloak — Authorization Server Open Source	46
18.1 Avvio Rapido con Docker	46
18.2 Configurazione via python-keycloak	46
18.3 Login Operatori HMI con Keycloak	47
Capitolo 19: OAuth 2.0 nei Microservizi	49
19.1 Pattern di Propagazione del Token	49
19.2 API Gateway con Validazione JWT Centralizzata	49

Capitolo 20: Argomenti Avanzati — PAR, DPoP, mTLS	51
20.1 PAR — Pushed Authorization Requests (RFC 9126)	51
20.2 DPoP — Demonstration of Proof of Possession (RFC 9449)	51
Capitolo 21: Conformita' e Standard Industriali	53
21.1 IEC 62443 e OAuth 2.0	53
21.2 Audit Trail con OAuth	53
21.3 Rotazione Automatica dei Segreti	54
Capitolo 22: OAuth 2.0 vs Alternative — SAML, API Key, Basic Auth	56
22.1 OAuth 2.0 vs SAML — Quando Scegliere Cosa	56
22.2 API Key — Quando Ha Ancora Senso	56
Capitolo 23: Casi Studio Completi	58
23.1 Caso Studio: Dashboard SCADA con SSO	58
23.2 Caso Studio: CI/CD Pipeline con GitHub OAuth	60
Approfondimento: Il Contesto Normativo Europeo di OAuth 2.0	61
Sicurezza by Design — Principi Fondamentali	61
Evoluzione Storica: Da OAuth 1.0 a OAuth 2.1	62
Confronto tra Authorization Server: Google, Okta, Auth0, Keycloak	62
WebAuthn e FIDO2 — Il Futuro dell'Autenticazione Forte	63
Guida Implementativa Completa	65
Progettare un Sistema OAuth 2.0 da Zero	65
Monitoraggio e Observability OAuth	66
Scenari di Errore e Recovery	66
Ecosistema Python per OAuth 2.0	68
Librerie Client OAuth	68
Librerie JWT	68

Strumenti di Debug OAuth	68
Deployment OAuth in Produzione	70
Alta Disponibilità dell'Authorization Server	70
Configurazione NGINX come Reverse Proxy OAuth	71
Kubernetes Deployment con Keycloak Operator	71
Analisi Tecnica degli RFC OAuth 2.0	73
RFC 6749 — Analisi Punto per Punto	73
RFC 6750 — Bearer Token Usage	73
RFC 7636 — PKCE in Dettaglio	74
RFC 8693 — Token Exchange	74
Scenari di Attacco Reali e Mitigazioni	76
Scenari Avanzati e Specializzati	78
OAuth 2.0 per Sistemi Multi-Tenant	78
OAuth 2.0 con gRPC e Protocol Buffers	79
OAuth 2.0 per MQTT e Protocolli IoT	80
Strumenti e Risorse per OAuth 2.0	82
Strumenti di Test e Debug	82
Risorse di Apprendimento e Riferimento	82
Checklist Finale Prima del Go-Live	82
Guida Enterprise e Governance OAuth 2.0	84
Governance e Processi per OAuth 2.0 Enterprise	84
Framework di Compliance e OAuth 2.0	84
OAuth Incident Response Playbook	85
Training e Cultura della Sicurezza OAuth	86
Roadmap Migrazione verso OAuth 2.1	86

FAQ e Pattern Avanzati	87
Domande Frequenti (FAQ)	87
Pattern Architetture Avanzati	89
Pattern: API Gateway con OAuth Sidecar	89
Pattern: BFF — Backend For Frontend	89
Pattern: Step-Up Authentication	89
Conclusioni e Riepilogo	91
I 10 Principi Fondamentali di OAuth 2.0	91
Il Futuro di OAuth 2.0	92
Appendice B: Esempi di Codice Aggiuntivi	93
B1 Validazione Token Completa con python-jose	93
B2 Client OAuth Asincrono con httpx	93
B3 Token Binding con Fingerprinting IP	94
Appendice C: Configurazioni di Riferimento	96
C1 Keycloak realm.json — Export di Configurazione	96
C2 .env e docker-compose per sviluppo locale	97
Appendice: Riferimento Rapido e Glossario	98
A1 RFC di Riferimento	98
A2 Quale Grant Type Scegliere?	98
A3 Glossario	98
A4 OAuth 2.1 — Le Novità Principali	99

Capitolo 1: L'Era Pre-OAuth — Il Problema delle Password

Per capire OAuth 2.0 occorre partire da dove tutto è cominciato: il caos dell'integrazione di servizi web tra il 2004 e il 2008. La storia di OAuth è la storia di come l'industria ha risolto uno dei problemi di sicurezza più pericolosi che essa stessa aveva creato.

1.1 Il Web 2.0 e l'Esplosione delle API

Tra il 2004 e il 2008 assistiamo all'esplosione del Web 2.0. I siti web non sono più isole isolate: diventano piattaforme con API pubbliche. Flickr lancia le sue API nel 2004, Amazon nel 2006, Google Calendar nel 2006, Twitter nel 2007, Facebook nel 2007. Improvvisamente un'app può leggere le foto di un utente su Flickr, aggiungere eventi al suo Google Calendar, twittare per suo conto. La domanda diventa: come autorizza l'utente queste operazioni?

1.2 L'Anti-Pattern della Password Sharing

La risposta ovvia — e terribile — fu: chiedere all'utente username e password, poi usarle al suo posto. Questo pattern è noto oggi come 'password anti-pattern'. Era così diffuso da essere considerato normale.

Esempio reale: Nel 2007 Yelp chiedeva agli utenti di inserire le credenziali Gmail direttamente nel sito Yelp per importare i contatti. L'utente non aveva modo di sapere cosa altro Yelp facesse con quelle credenziali, né modo di revocare l'accesso senza cambiare la password Gmail.

Perché questo era catastrofico:

- L'app di terze parti aveva accesso illimitato all'intero account
- Non esisteva modo di revocare l'accesso senza cambiare la password
- Se l'app veniva hackerata, le credenziali erano esposte
- Nessun audit trail: non sapevi cosa aveva fatto l'app
- Nessuna granularità: non potevi dire 'leggi solo le email, non inviarne'
- Il provider del servizio (Gmail) non sapeva che un terzo stava accedendo

1.3 I Tentativi Pre-OAuth e il Fallimento della Frammentazione

Il settore era consapevole del problema. Ogni provider inventò la propria soluzione: Google aveva AuthSub (2005), Yahoo aveva BBAuth, AOL aveva OpenAuth. Nessuna era compatibile con le altre. Gli sviluppatori di terze parti dovevano implementare un sistema diverso per ogni integrazione. Era insostenibile.

I requisiti che uno standard aperto doveva soddisfare erano chiari: delegazione senza password, granularità dei permessi, revocabilità, scadenza automatica dei token, audit trail, universalità e semplicità implementativa.

Soluzione	Anno	Limite principale
Google AuthSub	2005	Solo servizi Google, non standard

Flickr Auth API	2004	Solo Flickr, firma MD5 proprietaria
Yahoo BBAuth	2007	Solo Yahoo, non interoperabile
AOL OpenAuth	2007	Solo AOL, deprecato rapidamente
OAuth 1.0	2007	Firma HMAC complessa, no mobile native

Capitolo 2: Nascita di OAuth 1.0 (2007–2010)

Nel novembre 2006, Blaine Cook (lead developer di Twitter) e Larry Halff si incontrarono a una conferenza e avviarono la discussione di uno standard aperto per la delegazione di accesso. Nacque così il gruppo di lavoro che, con Chris Messina, David Recordon ed Eran Hammer-Lahav, avrebbe creato OAuth.

2.1 OAuth 1.0: Il Flusso Three-Legged

OAuth 1.0 (RFC 5849, 2010) introduce un meccanismo basato su firme crittografiche HMAC-SHA1. Il flusso principale coinvolge tre attori — l'utente, l'app client e il server OAuth — ed è detto perciò 'three-legged'.

Passo	Chi	Evento
1	App → Server	Richiede request_token temporaneo
2	Server → App	Emette request_token non ancora autorizzato
3	App → Browser	Reindirizza l'utente al server OAuth
4	Utente → Server	Si autentica e autorizza l'app
5	Server → Browser	Redirect al callback con oauth_verifier
6	App → Server	Scambia request_token + verifier per access_token
7	Server → App	Emette l'access token definitivo
8	App → API	Usa l'access token (con firma) per ogni richiesta

2.2 Il Problema della Firma e Perché Serve OAuth 2.0

La firma HMAC-SHA1 era il punto di forza ma anche il tallone d'Achille di OAuth 1.0. Per costruire la firma occorre: ordinare alfabeticamente i parametri, URL-encodarli in modo specifico, costruire la 'base string', calcolare HMAC-SHA1. Un carattere sbagliato rompeva tutto.

■ Firma OAuth 1.0 vs semplicità OAuth 2.0

```
# Firma OAuth 1.0 – mostra la complessità rispetto a OAuth 2.0
import hmac, hashlib, base64, urllib.parse, time, uuid
```

```
def oauth1_signature(method, url, params, consumer_secret, token_secret=""):
    oauth_params = {
        'oauth_consumer_key': 'your_consumer_key',
        'oauth_nonce': uuid.uuid4().hex,
        'oauth_signature_method': 'HMAC-SHA1',
        'oauth_timestamp': str(int(time.time())),
        'oauth_token': 'your_access_token',
        'oauth_version': '1.0',
    }
    all_params = {**params, **oauth_params}
    # Ordine alfabetico obbligatorio
    sorted_params = sorted(all_params.items())
    param_string = '&'.join(
        f"{urllib.parse.quote(k, safe='')}={urllib.parse.quote(v, safe='')}}"
        for k, v in sorted_params
```

```
)
base_string = '&'.join([
    method.upper(),
    urllib.parse.quote(url, safe=''),
    urllib.parse.quote(param_string, safe='')
])
signing_key = f"{urllib.parse.quote(consumer_secret, safe='')}&" f"{urllib.parse.quote(
hashed = hmac.new(signing_key.encode(), base_string.encode(), hashlib.sha1)
return base64.b64encode(hashed.digest()).decode()

# In OAuth 2.0 tutto ciò è sostituito da:
# headers = {'Authorization': f'Bearer {access_token}'}
# Ecco perché OAuth 2.0 era necessario.
```

OAuth 2.0 elimina completamente le firme: la sicurezza è delegata a HTTPS/TLS. Il risultato è un protocollo molto più semplice da implementare correttamente.

Capitolo 3: OAuth 2.0 — Framework, Attori e Token

Il 6 ottobre 2012 viene pubblicato RFC 6749 — 'The OAuth 2.0 Authorization Framework'. La parola chiave è 'Framework', non 'Protocollo': OAuth 2.0 definisce concetti e flussi, lasciando molti dettagli all'implementazione del provider.

3.1 I Quattro Attori

Attore	Ruolo	Esempio
Resource Owner	Chi possiede i dati (di solito l'utente)	Tu (l'utente)
Resource Server	Ospita le risorse protette, accetta access token	Google Photos API
Client	App che vuole accedere per conto del R.O.	PhotoPrinter App
Authorization Server	Autentica il R.O. ed emette i token	accounts.google.com

3.2 Access Token, Refresh Token e Authorization Code

Credenziale	Vita	Scopo	Chi lo vede
Authorization Code	10 minuti, uso singolo	Scambiato per token nel backend	Solo il backend
Access Token	1 ora tipico	Accedere alle API protette	Client + RS
Refresh Token	Giorni/settimane	Ottenere nuovi access token	Solo il client
ID Token (OIDC)	Breve	Identità dell'utente (JWT)	Client

3.3 Gli Scope — Granularità dei Permessi

Gli scope sono stringhe che definiscono l'ambito di accesso. Non esiste un standard unico per i nomi: ogni provider li definisce liberamente. OpenID Connect introduce scope standardizzati (openid, profile, email).

Provider	Scope	Permesso
Google	https://www.googleapis.com/auth/calendar.readonly	Leggi Calendar
Google	https://www.googleapis.com/auth/gmail.send	Invia email
GitHub	repo	Accesso completo ai repo
GitHub	user:email	Leggi email utente
Spotify	playlist-read-private	Leggi playlist private
OpenID Conn.	openid	Autenticazione OIDC
OpenID Conn.	profile	Nome, foto profilo
Industriale	telemetry:write	Scrivi dati telemetrici

Principio del minimo privilegio: richiedi sempre il numero minimo di scope necessari per la funzionalità dell'app. Scope ampi aumentano il rischio in caso di token rubato.

Capitolo 4: Authorization Code Flow

L'Authorization Code Flow è il flusso più usato e sicuro. È progettato per applicazioni con un backend server in grado di mantenere segreti. Comprendere questo flusso in dettaglio è essenziale per qualsiasi sviluppatore OAuth.

4.1 Registrazione dell'App

Prima di qualsiasi flusso OAuth, l'app deve essere registrata presso l'Authorization Server (es. console.cloud.google.com). La registrazione produce `client_id` (pubblico) e `client_secret` (segreto).

Parametro	Obbligatorio	Note
Application Name	Sì	Mostrato all'utente nella schermata di consenso
Redirect URI(s)	Sì	URL esatti a cui l'AS reindirizza dopo l'auth
Grant Types	Sì	Quali flussi OAuth l'app userà
Scope richiesti	Varia	Permessi che l'app richiederà
Privacy Policy URL	Raccomandato	Richiesto da molti provider

4.2 Il Flusso Passo per Passo

Step 1 — Costruisci l'URL di autorizzazione:

■ Step 1: URL autorizzazione

```
import urllib.parse, secrets
from flask import session, redirect

state = secrets.token_urlsafe(32) # Protezione CSRF
session['oauth_state'] = state

params = {
    'response_type': 'code',
    'client_id': 'YOUR_CLIENT_ID',
    'redirect_uri': 'https://myapp.com/callback',
    'scope': 'openid email profile',
    'state': state,
    'access_type': 'offline', # Google: chiedi il refresh_token
    'prompt': 'consent', # Google: mostra sempre il consenso
}

auth_url = 'https://accounts.google.com/o/oauth2/v2/auth?' + urllib.parse.urlencode(params)
return redirect(auth_url)
```

Step 2 — Callback: scambia il code per i token:

■ Step 2: Scambio code per token (backend)

```
import requests
from flask import request, session, redirect

def oauth_callback():
    # Verifica CSRF
    if request.args.get('state') != session.pop('oauth_state', None):
        return "CSRF attack rilevato!", 403
```

```

if 'error' in request.args:
    return f"Errore: {request.args['error']}", 400

# Scambia il code – questo avviene nel BACKEND (client_secret mai esposto)
resp = requests.post('https://oauth2.googleapis.com/token', data={
    'code': request.args['code'],
    'client_id': 'YOUR_CLIENT_ID',
    'client_secret': 'YOUR_CLIENT_SECRET', # MAI nel frontend!
    'redirect_uri': 'https://myapp.com/callback',
    'grant_type': 'authorization_code',
})
tokens = resp.json()

session['access_token'] = tokens['access_token']
session['refresh_token'] = tokens.get('refresh_token')
session['token_expires'] = time.time() + tokens.get('expires_in', 3600)
return redirect('/dashboard')

```

Step 3 — Usa il token con Bearer:

■ Step 3: Chiamata API con Bearer token

```

def call_api(access_token):
    resp = requests.get(
        'https://www.googleapis.com/oauth2/v3/userinfo',
        headers={'Authorization': f'Bearer {access_token}'},
        # 'Bearer' è obbligatorio – definito in RFC 6750
    )
    if resp.status_code == 401:
        return None, 'token_expired' # Gestisci il refresh
    return resp.json(), None

user, err = call_api(session['access_token'])
print(f"Ciao {user['name']} ({user['email']})")

```

4.3 Risposta Token — Analisi Completa

■ Risposta JSON del token endpoint

```

{
  "access_token": "ya29.a0AfH6SMBx...", // Token per le API (opaco per il client)
  "expires_in": 3599, // Scade tra N secondi (max 1 ora tipico)
  "refresh_token": "1//0eDVQZ...", // Per rinnovare (solo se offline/consent)
  "scope": "openid email profile", // Scope effettivamente concessi
  "token_type": "Bearer", // Tipo token (sempre Bearer in OAuth 2.0)
  "id_token": "eyJhbGciOiJIUzI1NiIs... // JWT solo con scope openid (OIDC)
}

```

Capitolo 5: PKCE — Proof Key for Code Exchange

PKCE (RFC 7636, pronunciato 'Pixy') aggiunge un livello di sicurezza all'Authorization Code Flow per applicazioni che non possono mantenere un `client_secret` in modo sicuro: app mobile, SPA, CLI tools.

5.1 Il Problema: Authorization Code Interception

Su app mobile, dove metti il `client_secret`? Chiunque decompila l'APK/IPA può estrarlo. Senza `client_secret`, un'app malevola registrata con lo stesso custom URI scheme può intercettare il codice e scambiarlo per un token. Questo attacco si chiama 'Authorization Code Interception Attack'.

■ Senza PKCE, un'app malevola che intercetta l'authorization code può ottenere l'access token al posto tuo, anche senza conoscere il `client_secret`.

5.2 Meccanismo PKCE — Code Verifier e Code Challenge

Prima della richiesta, il client genera un segreto casuale (`code_verifier`) e ne calcola l'hash SHA-256 (`code_challenge`). Invia solo l'hash nell'URL. Al momento dello scambio del code, invia il verifier originale: l'AS verifica che `SHA256(verifier) == challenge`. L'attaccante che ha intercettato l'URL non conosce il verifier originale.

■ PKCE completo — client pubblico senza `client_secret`

```
import secrets, hashlib, base64, urllib.parse, requests

class PKCEFlow:
    def __init__(self, client_id, redirect_uri, auth_url, token_url, scope):
        self.client_id = client_id
        self.redirect_uri = redirect_uri
        self.auth_url = auth_url
        self.token_url = token_url
        self.scope = scope

    def get_auth_url(self):
        # 1. Genera code_verifier (segreto, 43-128 char URL-safe)
        self.code_verifier = secrets.token_urlsafe(32) # 43 caratteri
        # 2. Calcola code_challenge = BASE64URL(SHA256(verifier))
        digest = hashlib.sha256(self.code_verifier.encode()).digest()
        self.code_challenge = base64.urlsafe_b64encode(digest).rstrip(b'=').decode()
        self.state = secrets.token_urlsafe(16)

        params = {
            'response_type': 'code',
            'client_id': self.client_id,
            'redirect_uri': self.redirect_uri,
            'scope': self.scope,
            'state': self.state,
            'code_challenge': self.code_challenge, # Hash inviato nell'URL
            'code_challenge_method': 'S256',      # SHA-256
        }
        return f"{self.auth_url}?{urllib.parse.urlencode(params)}"
```



```
def exchange_code(self, auth_code, received_state):
    if received_state != self.state:
        raise ValueError("State mismatch – possibile CSRF!")
    resp = requests.post(self.token_url, data={
        'grant_type': 'authorization_code',
        'code': auth_code,
        'redirect_uri': self.redirect_uri,
        'client_id': self.client_id,
        # Nessun client_secret! PKCE lo sostituisce per i client pubblici
        'code_verifier': self.code_verifier, # Segreto originale
    })
    return resp.json()

# Utilizzo
pkce = PKCEFlow('my_mobile_app', 'myapp://callback',
                'https://as.example.com/auth',
                'https://as.example.com/token', 'openid email')
auth_url = pkce.get_auth_url()
print(f"Apri nel browser: {auth_url}")
# ... dopo il redirect:
# tokens = pkce.exchange_code(code, received_state)
```

5.3 Perché PKCE è Ora per Tutti

✓ OAuth 2.1 rende PKCE obbligatorio per TUTTI i client, incluse le web app con backend. Implementando PKCE oggi, la tua applicazione è già conforme allo standard futuro.

Capitolo 6: Gli Altri Grant Type

OAuth 2.0 definisce quattro grant type principali. Scegliere quello sbagliato può compromettere la sicurezza dell'applicazione.

6.1 Client Credentials — Machine-to-Machine

Il Client Credentials Flow è usato quando l'applicazione accede a risorse proprie, non per conto di un utente. È il flusso tipico per microservizi, IoT, PLC e processi batch.

■ Client Credentials — PLC/IoT

```
import requests, time

class ClientCredentialsAuth:
    """Gestisce token OAuth per sistemi automatici (PLC, microservizi, IoT)."""

    def __init__(self, token_url, client_id, client_secret, scope):
        self.token_url = token_url
        self.client_id = client_id
        self.client_secret = client_secret
        self.scope = scope
        self._token = None
        self._expiry = 0

    def get_token(self):
        if self._token and time.time() < self._expiry - 30:
            return self._token # Ancora valido
        resp = requests.post(self.token_url,
                             data={'grant_type': 'client_credentials', 'scope': self.scope},
                             auth=(self.client_id, self.client_secret), timeout=10)
        resp.raise_for_status()
        d = resp.json()
        self._token = d['access_token']
        self._expiry = time.time() + d.get('expires_in', 3600)
        return self._token

    def headers(self):
        return {'Authorization': f'Bearer {self.get_token()}}'}

# Caso d'uso: PLC impianto trattamento acque che invia telemetria al cloud
auth = ClientCredentialsAuth(
    token_url='https://auth.factory.com/oauth2/token',
    client_id='plc_station_01',
    client_secret='plc_secret_from_env',
    scope='telemetry:write alarms:create'
)

# Invio dati telemetrici
resp = requests.post('https://api.factory.com/telemetry', json={
    'station': 'ST-01',
    'flow_rate': 125.3, # m3/h - portata impianto acque
    'ph': 7.1,
    'turbidity': 0.8, # NTU
    'cl_residual': 1.2, # ppm
}, headers=auth.headers())
```

```
print(f"Telemetria inviata: {resp.status_code}")
```

6.2 Device Authorization Grant (Device Flow)

Il Device Flow (RFC 8628) è progettato per dispositivi senza browser o tastiera: smart TV, console di gioco, dispositivi IoT, CLI tools. L'utente autentica su un secondo dispositivo (smartphone/PC).

■ Device Flow — CLI/IoT senza browser

```
import requests, time

def device_flow_login(client_id, scope='repo user:email'):
    # Step 1: richiedi device_code e user_code
    r = requests.post('https://github.com/login/device/code',
                      data={'client_id': client_id, 'scope': scope},
                      headers={'Accept': 'application/json'})
    d = r.json()

    print(f"Vai a: {d['verification_uri']}")
    print(f"Inserisci il codice: {d['user_code']}")    # Es. WDJB-MJHT
    print(f"Hai {d['expires_in']} secondi.")

    # Step 2: poll fino alla conferma
    interval = d['interval']    # Secondi tra i tentativi (tipico: 5)
    deadline = time.time() + d['expires_in']

    while time.time() < deadline:
        time.sleep(interval)
        r = requests.post('https://github.com/login/oauth/access_token',
                          data={'client_id': client_id,
                                'device_code': d['device_code'],
                                'grant_type': 'urn:ietf:params:oauth:grant-type:device_code'},
                          headers={'Accept': 'application/json'})
        t = r.json()
        if 'access_token' in t:
            return t['access_token']
        if t.get('error') == 'slow_down':
            interval += 5    # Il server chiede di rallentare
        elif t.get('error') not in ('authorization_pending',):
            raise Exception(t.get('error'))

    return None    # Timeout

token = device_flow_login('YOUR_GITHUB_CLIENT_ID')
if token:
    print(f"Autenticato! Token: {token[:15]}...")
```

6.3 Implicit e ROPC — Flussi Deprecati

■ Implicit Flow e ROPC (Resource Owner Password Credentials) sono entrambi deprecati in OAuth 2.1. Non usarli in nuove applicazioni. Implicit: token nell'URL, vulnerabile. ROPC: l'app vede la password, vanifica il punto di OAuth.

Capitolo 7: JWT — JSON Web Token

JWT (RFC 7519) è il formato più comune per gli access token in OAuth 2.0. Non è parte obbligatoria di OAuth 2.0, ma è il formato standard de facto e l'unico formato per gli ID token in OpenID Connect.

7.1 Struttura: header.payload.signature

Un JWT è composto da tre parti separate da punti, ognuna codificata in Base64URL:

■ Struttura JWT — le tre parti

```
# HEADER: algoritmo e tipo
{
  "alg": "RS256",    # Algoritmo di firma (RSA + SHA-256)
  "typ": "JWT",      # Tipo di token
  "kid": "key-001"   # Key ID (quale chiave pubblica usare per verificare)
}

# PAYLOAD: i claims (affermazioni)
{
  # Claims standard RFC 7519
  "iss": "https://accounts.google.com", # Issuer
  "sub": "user_123456",                 # Subject (ID utente)
  "aud": "my_app_client_id",            # Audience (per chi e' il token)
  "exp": 1716969600,                    # Expiration (Unix timestamp)
  "iat": 1716966000,                    # Issued At
  "nbf": 1716966000,                    # Not Before
  "jti": "unique-token-id-abc123",      # JWT ID (univoco, anti-replay)
  # Claims personalizzati
  "email": "mario@example.com",
  "scope": "openid email profile",
  "roles": ["operatore", "supervisore"]
}

# SIGNATURE: firma crittografica
# RS256: RSASSA-PKCS1-v1_5(SHA256(base64url(header) + '.' + base64url(payload)))
# Verifica: chiunque con la chiave pubblica dell'AS puo' verificarla
```

7.2 Algoritmi di Firma

Algoritmo	Tipo	Raccomandato per	Note
HS256	HMAC-SHA256	App single-tenant	Chiave simmetrica condivisa
RS256	RSA-SHA256	OAuth/OIDC (chiave pubblica)	Il più usato in produzione
ES256	ECDSA-SHA256	Mobile, IoT	Chiavi piccole, veloce
PS256	RSA-PSS	FIPS compliance	Più sicuro di RS256
none	Nessuna	MAI IN PRODUZIONE	VULNERABILITA' CRITICA

7.3 Validazione Corretta di un JWT

■ Non leggere MAI l'algoritmo dall'header del JWT e usarlo per la validazione: un attaccante può impostare 'alg':'none' o 'alg':'HS256' con la chiave pubblica. Specifica tu quale algoritmo ti aspetti.

■ Validazione JWT sicura con PyJWT

```
# pip install PyJWT cryptography
import jwt, requests

def validate_jwt(token, expected_issuer, expected_audience, jwks_uri):
    """Validazione JWT sicura con tutte le verifiche necessarie."""
    # 1. Non leggere alg dall'header – specificalo tu
    ALLOWED_ALGOS = ['RS256']

    # 2. Recupera JWKS (in produzione: metti in cache con TTL)
    jwks = requests.get(jwks_uri).json()

    # 3. Decodifica e verifica
    try:
        payload = jwt.decode(
            token,
            jwks,
            algorithms=ALLOWED_ALGOS,          # FISSO, non dall'header
            audience=expected_audience,
            issuer=expected_issuer,
            options={
                'verify_exp': True,             # Controlla scadenza
                'verify_iss': True,             # Controlla issuer
                'verify_aud': True,             # Controlla audience
                'require': ['exp', 'iss', 'sub'], # Claims obbligatori
            }
        )
        return payload
    except jwt.ExpiredSignatureError:
        raise ValueError("Token scaduto")
    except jwt.InvalidSignatureError:
        raise ValueError("Firma non valida – token contraffatto?")
    except jwt.InvalidIssuerError:
        raise ValueError(f"Issuer non valido")
    except jwt.DecodeError as e:
        raise ValueError(f"JWT malformato: {e}")

# Utilizzo
try:
    claims = validate_jwt(
        token=access_token,
        expected_issuer='https://accounts.google.com',
        expected_audience='my_client_id',
        jwks_uri='https://www.googleapis.com/oauth2/v3/certs'
    )
    print(f"Token valido per: {claims['email']}")
except ValueError as e:
    print(f"Token non valido: {e}")
```

Capitolo 8: OpenID Connect (OIDC)

OpenID Connect è uno strato di autenticazione costruito sopra OAuth 2.0. OAuth 2.0 gestisce l'autorizzazione (cosa puoi fare); OIDC aggiunge l'autenticazione (chi sei).

8.1 OAuth 2.0 vs OpenID Connect

Aspetto	OAuth 2.0	OpenID Connect
Scopo	Autorizzazione	Autenticazione
Domanda	Cosa può fare l'app?	Chi è l'utente?
Token chiave	Access Token (opaco)	ID Token (JWT)
Info utente	Non standardizzata	UserInfo endpoint standard
Scope base	Definiti dal provider	openid (obbligatorio)
Discovery	Non standard	/.well-known/openid-configuration

Errore comune: usare OAuth 2.0 per autenticare gli utenti. Avere un access token non dice chi è l'utente — dice solo che un'app ha permesso di accedere a certe risorse. Per l'autenticazione usa OIDC con ID token.

8.2 Discovery Endpoint e Auto-Configurazione

■ OIDC Discovery

```
import requests

def get_oidc_config(issuer_url):
    """
    Recupera automaticamente la configurazione di qualsiasi provider OIDC.
    Funziona con Google, Microsoft, Auth0, Keycloak, Okta, ecc.
    """
    url = f"{issuer_url}/.well-known/openid-configuration"
    cfg = requests.get(url).json()
    print(f"Authorization: {cfg['authorization_endpoint']}")
    print(f"Token: {cfg['token_endpoint']}")
    print(f"UserInfo: {cfg['userinfo_endpoint']}")
    print(f"JWKS: {cfg['jwks_uri']}")
    print(f"Revocation: {cfg.get('revocation_endpoint', 'N/A')}")
    return cfg

# Esempi con provider reali
cfg_google = get_oidc_config('https://accounts.google.com')
cfg_ms = get_oidc_config('https://login.microsoftonline.com/common/v2.0')
# cfg_kc = get_oidc_config('https://keycloak.factory.com/realms/myrealm')
```

8.3 Login con Google — Implementazione Flask Completa

■ Login Google con OIDC — Flask completo

```
from flask import Flask, redirect, request, session
import requests, jwt as pyjwt, secrets, urllib.parse
```

```

app = Flask(__name__)
app.secret_key = secrets.token_hex(32)

GOOGLE_CLIENT_ID      = 'YOUR_CLIENT_ID'
GOOGLE_CLIENT_SECRET  = 'YOUR_CLIENT_SECRET'
REDIRECT_URI          = 'http://localhost:5000/callback'
OIDC = requests.get(
    'https://accounts.google.com/.well-known/openid-configuration').json()

@app.route('/login')
def login():
    state = secrets.token_urlsafe(32)
    nonce = secrets.token_urlsafe(32) # Anti-replay per l'ID token
    session.update({'oauth_state': state, 'oauth_nonce': nonce})
    params = {
        'response_type': 'code',
        'client_id':     GOOGLE_CLIENT_ID,
        'redirect_uri':  REDIRECT_URI,
        'scope':         'openid email profile', # 'openid' attiva OIDC
        'state':         state,
        'nonce':         nonce,
    }
    return redirect(f"{OIDC['authorization_endpoint']}?{urllib.parse.urlencode(params)}")

@app.route('/callback')
def callback():
    if request.args.get('state') != session.pop('oauth_state', None):
        return "CSRF!", 403
    # Scambia code per token
    tokens = requests.post(OIDC['token_endpoint'], data={
        'code': request.args['code'], 'grant_type': 'authorization_code',
        'client_id': GOOGLE_CLIENT_ID, 'client_secret': GOOGLE_CLIENT_SECRET,
        'redirect_uri': REDIRECT_URI,
    }).json()
    # Verifica ID token
    jwks = requests.get(OIDC['jwks_uri']).json()
    hdr = pyjwt.get_unverified_header(tokens['id_token'])
    from jwt.algorithms import RSAAlgorithm
    pub_key = RSAAlgorithm.from_jwk(
        next(k for k in jwks['keys'] if k['kid'] == hdr['kid']))
    claims = pyjwt.decode(tokens['id_token'], pub_key, algorithms=['RS256'],
        audience=GOOGLE_CLIENT_ID,
        issuer='https://accounts.google.com')
    if claims.get('nonce') != session.pop('oauth_nonce', None):
        return "Nonce mismatch!", 403
    session['user'] = {'email': claims['email'], 'name': claims.get('name')}
    return redirect('/')

@app.route('/')
def home():
    user = session.get('user')
    if not user: return '<a href="/login">Login con Google</a>'
    return f"Ciao {user['name']}! <a href='/logout">Logout</a>"

@app.route('/logout')
def logout():
    session.clear()
    return redirect('/')

```

Capitolo 9: Refresh Token, Revoca e Ciclo di Vita

La gestione del ciclo di vita dei token è uno degli aspetti più importanti e spesso trascurati nell'implementazione OAuth 2.0.

9.1 Refresh Token Flow

■ Token Manager con refresh e revoca

```
import requests, time
from threading import Lock

class TokenManager:
    """Token manager thread-safe con refresh automatico."""

    def __init__(self, token_url, client_id, client_secret):
        self.token_url = token_url
        self.client_id = client_id
        self.client_secret = client_secret
        self._access_token = None
        self._refresh_token = None
        self._expiry = 0
        self._lock = Lock()

    def set_initial_tokens(self, access_token, refresh_token, expires_in):
        with self._lock:
            self._access_token = access_token
            self._refresh_token = refresh_token
            self._expiry = time.time() + expires_in - 30

    def _do_refresh(self):
        if not self._refresh_token:
            raise ValueError("Nessun refresh token – riautentica l'utente")
        resp = requests.post(self.token_url, data={
            'grant_type': 'refresh_token',
            'refresh_token': self._refresh_token,
            'client_id': self.client_id,
            'client_secret': self.client_secret,
        })
        if resp.status_code == 400:
            err = resp.json().get('error', '')
            if err == 'invalid_grant':
                raise ValueError("Refresh token scaduto/revocato – riautentica")
            raise Exception(f"Refresh fallito: {resp.text}")
        d = resp.json()
        self._access_token = d['access_token']
        self._expiry = time.time() + d.get('expires_in', 3600) - 30
        if 'refresh_token' in d:
            self._refresh_token = d['refresh_token'] # Rotation

    def get_valid_token(self):
        with self._lock:
            if time.time() >= self._expiry:
                self._do_refresh()
```



```

        return self._access_token

    def revoke(self, revoke_url):
        """Revoca al logout – RFC 7009."""
        if self._refresh_token:
            requests.post(revoke_url, data={
                'token': self._refresh_token,
                'token_type_hint': 'refresh_token',
                'client_id': self.client_id,
                'client_secret': self.client_secret,
            })
        self._access_token = self._refresh_token = None

```

9.2 Token Introspection — RFC 7662

Il Resource Server può verificare un token opaco chiamando l'endpoint di introspection dell'AS.

■ Token Introspection

```

def introspect_token(token, introspect_url, client_id, client_secret):
    resp = requests.post(introspect_url,
        data={'token': token, 'token_type_hint': 'access_token'},
        auth=(client_id, client_secret))
    result = resp.json()
    if not result.get('active'):
        return None # Token non attivo o scaduto
    print(f"Attivo – sub:{result.get('sub')} scope:{result.get('scope')}")
    return result

# Risposta tipica:
# {"active":true,"sub":"user_123","scope":"read write","exp":1716969600}

```

9.3 Best Practice per lo Storage dei Token

Contesto	Storage raccomandato	Da evitare
Web app (backend)	Cookie HttpOnly+Secure	localStorage, URL
SPA	Memory (variabile JS)	localStorage, cookie senza HttpOnly
App mobile iOS	Keychain	NSUserDefaults, file plain
App mobile Android	EncryptedSharedPrefs, Keystore	SharedPreferences plain
Server/CLI	Variabile d'ambiente, vault	File .env in Git, hardcoded
PLC/IoT	Secure enclave, TPM	EEPROM non cifrata

Capitolo 10: Sicurezza OAuth 2.0 — Attacchi e Difese

OAuth 2.0 è sicuro se implementato correttamente. La maggior parte delle vulnerabilità non riguarda il protocollo, ma l'implementazione. Questa sezione copre gli attacchi più comuni.

10.1 Tabella degli Attacchi Principali

Attacco	Vettore	Difesa
CSRF	State parameter mancante	State casuale, verifica sempre
Code Interception	redirect_uri con custom scheme	PKCE obbligatorio
Open Redirect	redirect_uri non validato	Whitelist esatta URI
Token Leakage via Referer	Token nell'URL	Token solo in header
JWT alg:none	Header alg modificato	Specifica alg atteso — fisso
JWT Algorithm Confusion	RS256 confuso con HS256	Specifica algoritmi attesi
Replay Attack	Token riusato	Nonce (OIDC), exp breve, jti
Mix-Up Attack	AS malevolo con stesso client_id	Verifica iss nel token
Clickjacking su AS	AS in iframe	X-Frame-Options: DENY sull'AS
Client Impersonation	client_id pubblico noto	PKCE, client auth

10.2 JWT Algorithm Confusion — Dettaglio

■ Algorithm Confusion — vulnerabilità e fix

```
# VULNERABILE: legge alg dall'header non verificato
def validate_UNSAFE(token, public_key, secret):
    alg = jwt.get_unverified_header(token)['alg'] # Controllato dall'attaccante!
    key = public_key if alg == 'RS256' else secret # Con HS256 usa la pubkey come secret!
    return jwt.decode(token, key, algorithms=[alg])

# SICURO: specifica tu l'algoritmo atteso
def validate_SAFE(token, public_key):
    ALLOWED = ['RS256'] # Fisso nel codice, non nell'header del token
    return jwt.decode(token, public_key, algorithms=ALLOWED,
                      options={'require': ['exp', 'iss', 'aud', 'sub']})
```

10.3 Checklist di Sicurezza OAuth 2.0

- ✓ **HTTPS ovunque:** Tutti gli endpoint OAuth — mai HTTP in produzione
- ✓ **State parameter:** Sempre presente, casuale (≥ 32 byte), verificato
- ✓ **PKCE:** Obbligatorio per client pubblici, raccomandato per tutti
- ✓ **redirect_uri whitelist:** Match esatto, non prefix/regex — un carattere sbagliato = attacco
- ✓ **Access token breve:** Max 1 ora; più corto = meno esposizione se rubato
- ✓ **Refresh token rotation:** Nuovo RT ad ogni utilizzo; rileva replay se vecchio RT usato

- ✓ **Revoca al logout:** Chiama revocation endpoint ogni volta
- ✓ **Algoritmo JWT fisso:** Non leggere alg dall'header — usa whitelist nel codice
- ✓ **Validazione claims JWT:** Verifica iss, aud, exp, nbf, iat — obbligatori tutti
- ✓ **Secure storage:** HttpOnly cookie o Keychain/Keystore — mai localStorage
- ✓ **Scope minimale:** Principio del minimo privilegio — chiedi solo ciò che serve
- ✓ **Nonce per OIDC:** Previene replay attack sull'ID token

Capitolo 11: OAuth 2.0 nell'Automazione Industriale

I sistemi di automazione industriale — PLC, SCADA, HMI, robot — stanno integrandosi sempre più con cloud e sistemi IT. OAuth 2.0 è il protocollo standard per questa integrazione sicura, ma presenta sfide specifiche.

11.1 Sfide dell'Automazione Industriale

- Risorse limitate: alcuni PLC hanno poca memoria/CPU per crittografia
- Connettività intermittente: reti OT non sempre connesse a Internet
- Cicli di vita lunghi: dispositivi che durano 20+ anni
- Latenza critica: validazione JWT non deve impattare i tempi di ciclo
- Sistemi eterogenei: Yaskawa, Siemens, Rockwell usano protocolli diversi
- Sicurezza fisica: token non devono permettere accesso fisico senza ulteriori verifiche

11.2 Pattern: Edge OAuth Proxy per PLC

Per PLC con risorse limitate, il pattern consigliato è un 'Edge OAuth Proxy': un gateway locale (Raspberry Pi, industrial PC) che gestisce OAuth per conto dei PLC. Il PLC comunica in rete locale trusted con il proxy; il proxy autentica verso il cloud con Client Credentials.

■ Edge OAuth Proxy per PLC e robot Yaskawa

```
# Edge OAuth Proxy – PLC locale -> cloud con OAuth
from flask import Flask, request, jsonify
import requests, time, os

app = Flask(__name__)

class EdgeProxy:
    """Proxy OAuth per dispositivi industriali che non supportano OAuth nativamente."""
    def __init__(self, token_url, client_id, client_secret, cloud_api, scope):
        self.auth = ClientCredentialsAuth(token_url, client_id, client_secret, scope)
        self.cloud_api = cloud_api

    def forward(self, endpoint, payload):
        resp = requests.post(
            f"{self.cloud_api}{endpoint}",
            json=payload,
            headers={**self.auth.headers(),
                    'X-Station-ID': os.environ.get('STATION_ID', 'ST-01')},
            timeout=5)
        return resp.status_code, resp.json() if resp.content else {}

proxy = EdgeProxy(
    token_url = 'https://auth.factory.com/oauth2/token',
    client_id  = 'edge_proxy_st01',
    client_secret = os.environ['PROXY_SECRET'],
    cloud_api  = 'https://api.factory.com',
    scope      = 'telemetry:write alarms:create'
)

@app.route('/api/telemetry', methods=['POST'])
```

```
def recv_telemetry():
    data = request.json
    # Validazione token locale semplice (PLC e' su rete trusted)
    if data.get('plc_token') != os.environ.get(f"PLC_TOKEN_{data.get('station_id','')}"):
        return jsonify({'error': 'unauthorized'}), 401
    status, result = proxy.forward('/telemetry', {
        'station': data['station_id'],
        'timestamp': time.time(),
        'ph': data.get('ph'),
        'flow_m3h': data.get('flow_m3h'),
        'turbidity': data.get('turbidity_ntu'),
        'cl_ppm': data.get('cl_ppm'),
        'robot_state': data.get('robot_yaskawa'), # Stato robot Yaskawa
    })
    return jsonify({'cloud': status, **result})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

11.3 Esempio: Impianto Trattamento Acque

■ SCADA impianto acque + Yaskawa con OAuth

Sistema completo: PLC Siemens S7-1500 + Robot Yaskawa
 # Edge proxy autenticato con OAuth Client Credentials

```
import requests, time, json
```

```
class ImpiantoAcqueClient:
    """
    Client OAuth per sistema SCADA impianto trattamento acque.
    Usa Client Credentials (machine-to-machine, nessun utente).
    """
    SCOPES = 'telemetry:write alarms:create robot:read setpoints:write'

    def __init__(self, oauth_config, api_base):
        self.auth = ClientCredentialsAuth(**oauth_config, scope=self.SCOPES)
        self.api_base = api_base

    def _h(self): return self.auth.headers()

    def send_cycle_data(self, station_id, measurements: dict):
        """Invia dati di ciclo con autenticazione OAuth."""
        return requests.post(f"{self.api_base}/telemetry/{station_id}",
            json={'ts': time.time(), **measurements}, headers=self._h(), timeout=3)

    def send_alarm(self, code, severity, description):
        """Invia allarme. severity: CRITICAL | HIGH | MEDIUM | LOW"""
        return requests.post(f"{self.api_base}/alarms",
            json={'code': code, 'severity': severity, 'desc': description,
                'ts': time.time()}, headers=self._h(), timeout=3)

    def get_robot_status(self, robot_id):
        """Leggi stato robot Yaskawa tramite API cloud."""
        r = requests.get(f"{self.api_base}/robots/{robot_id}/status",
            headers=self._h(), timeout=2)
        return r.json() if r.ok else None

    def set_dosage(self, chemical, dose_ppm):
```

```
        """Imposta dosaggio chimico (richiede scope setpoints:write)."""
        return requests.put(f"{self.api_base}/setpoints/dosage",
                            json={'chemical': chemical, 'dose_ppm': dose_ppm,
                                  'authorized_by': 'scada_system'},
                            headers=self._h(), timeout=5)

# Ciclo di controllo (eseguito ogni 5 secondi)
client = ImpiantoAcqueClient(
    oauth_config={'token_url': 'https://auth.factory.com/oauth2/token',
                  'client_id': 'scada_impianto', 'client_secret': '****'},
    api_base='https://api.factory.com'
)
while True:
    # Leggi dati dal PLC via OPC-UA o Modbus (non mostrato)
    plc_data = read_plc_registers() # stub
    client.send_cycle_data('ST-01', {
        'ph': plc_data.ph, 'flow': plc_data.flow_m3h,
        'turbidity': plc_data.ntu, 'cl': plc_data.cl_ppm
    })
    if plc_data.ph < 6.5 or plc_data.ph > 8.5:
        client.send_alarm('PH_OUT_OF_RANGE', 'HIGH',
                          f'pH={plc_data.ph} fuori range 6.5-8.5')
    robot = client.get_robot_status('ROBOT_YRC1000_01')
    if robot and robot.get('alarm_code'):
        client.send_alarm(f"ROBOT_{robot['alarm_code']}", 'HIGH',
                          f"Allarme robot: {robot.get('alarm_description')}")
    time.sleep(5)
```

Capitolo 12: Provider Reali e Librerie Python

Questa sezione raccoglie esempi pratici con i provider OAuth più usati e le principali librerie Python.

12.1 Google — google-auth e Workspace API

■ Google OAuth con API client

```
# pip install google-auth google-auth-oauthlib google-api-python-client
from google_auth_oauthlib.flow import InstalledAppFlow
from google.auth.transport.requests import Request
from googleapiclient.discovery import build
import pickle, os

SCOPES = ['https://www.googleapis.com/auth/gmail.readonly',
          'https://www.googleapis.com/auth/calendar.readonly']

def get_google_creds(creds_file='credentials.json', token_file='token.pkl'):
    creds = None
    if os.path.exists(token_file):
        with open(token_file, 'rb') as f: creds = pickle.load(f)
    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
    elif not creds or not creds.valid:
        flow = InstalledAppFlow.from_client_secrets_file(creds_file, SCOPES)
        creds = flow.run_local_server(port=0)
    with open(token_file, 'wb') as f: pickle.dump(creds, f)
    return creds

creds = get_google_creds()
gmail = build('gmail', 'v1', credentials=creds)
msgs = gmail.users().messages().list(userId='me', maxResults=5).execute()
print(f"Email trovate: {msgs.get('resultSizeEstimate')}")
```

12.2 Microsoft Azure AD con MSAL

■ Microsoft Azure AD con MSAL

```
# pip install msal requests
import msal, requests, os

app = msal.ConfidentialClientApplication(
    client_id = os.environ['AZURE_CLIENT_ID'],
    authority = f"https://login.microsoftonline.com/{os.environ['AZURE_TENANT_ID']}",
    client_credential = os.environ['AZURE_CLIENT_SECRET'],
)

# Client Credentials (M2M)
result = app.acquire_token_for_client(
    scopes=['https://graph.microsoft.com/.default'])
if 'access_token' in result:
    users = requests.get('https://graph.microsoft.com/v1.0/users',
        headers={'Authorization': f"Bearer {result['access_token']}"})
    print(f"Utenti: {len(users.get('value', []))}")
else:
    print(f"Errore: {result.get('error_description')}")
```

12.3 GitHub OAuth — requests-oauthlib

■ GitHub OAuth con requests-oauthlib

```
# pip install requests requests-oauthlib
from requests_oauthlib import OAuth2Session
import os

CLIENT_ID    = os.environ['GITHUB_CLIENT_ID']
CLIENT_SECRET= os.environ['GITHUB_CLIENT_SECRET']
REDIRECT_URI = 'http://localhost:5000/callback'

# Step 1: ottieni authorization URL
gh = OAuth2Session(CLIENT_ID, redirect_uri=REDIRECT_URI,
                  scope=['repo', 'user:email'])
auth_url, state = gh.authorization_url(
    'https://github.com/login/oauth/authorize')
print(f"Apri: {auth_url}")

# Step 2: dopo il redirect, scambia code per token
# os.environ['OAUTHLIB_INSECURE_TRANSPORT'] = '1' # Solo per sviluppo locale!
# token = gh.fetch_token('https://github.com/login/oauth/access_token',
#                       client_secret=CLIENT_SECRET, authorization_response=callback_url)

# Step 3: usa la sessione autenticata
# repos = gh.get('https://api.github.com/user/repos').json()
```

12.4 FastAPI Resource Server — Protezione Endpoint

■ FastAPI Resource Server con scope

```
# pip install fastapi uvicorn PyJWT cryptography
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt, requests

app = FastAPI()
bearer = HTTPBearer()
JWKS_URI = 'https://as.example.com/.well-known/jwks.json'
ISSUER = 'https://as.example.com'
AUDIENCE = 'my_api'

def get_current_user(creds: HTTPAuthorizationCredentials = Depends(bearer)) -> dict:
    token = creds.credentials
    try:
        jwks = requests.get(JWKS_URI).json() # In produzione: cache con TTL!
        claims = jwt.decode(token, jwks, algorithms=['RS256'],
                           audience=AUDIENCE, issuer=ISSUER)
        return claims
    except jwt.ExpiredSignatureError:
        raise HTTPException(401, "Token scaduto")
    except jwt.InvalidTokenError as e:
        raise HTTPException(401, f"Token non valido: {e}")

def require_scope(scope: str):
    def checker(user: dict = Depends(get_current_user)):
        if scope not in user.get('scope', '').split():
            raise HTTPException(403, f"Scope '{scope}' richiesto")
        return user
```



```
        return checker

@app.get('/api/telemetry')
async def get_telemetry(user=Depends(require_scope('telemetry:read'))):
    return {'data': [1,2,3], 'user': user['sub']}

@app.get('/api/admin')
async def admin(user=Depends(require_scope('admin'))):
    return {'message': 'Accesso admin', 'user': user['sub']}
```

Capitolo 13: HTTP e TLS — Fondamenta di OAuth 2.0

OAuth 2.0 è costruito interamente su HTTP/HTTPS. Comprendere i meccanismi HTTP sottostanti è indispensabile per implementare OAuth correttamente: header, status code, metodi, sicurezza TLS.

13.1 Anatomia di una Richiesta OAuth

■ Request/Response HTTP al token endpoint

```
# Richiesta tipica al Token Endpoint (RFC 6749)
POST /oauth2/token HTTP/1.1
Host: as.example.com
Content-Type: application/x-www-form-urlencoded
Authorization: Basic base64(client_id:client_secret)
Accept: application/json
Cache-Control: no-cache

grant_type=authorization_code&
code=SpLxl0BeZQQYbYS6WxSbIA&
redirect_uri=https%3A%2F%2Fmyapp.com%2Fcallback&
code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk

# Risposta
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store          # RFC 6749: OBBLIGATORIO
Pragma: no-cache                # RFC 6749: OBBLIGATORIO
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000

{
  "access_token": "mF_9.B5f-4.1JqM",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "tGzv3J0kF0XG5Qx2TLKWIA"
}
```

13.2 HTTP Status Code in OAuth

Status	Significato OAuth
200 OK	Token emesso con successo
302	Redirect al client dopo autorizzazione (con code o error)
400	Parametri mancanti o malformati — body JSON con error
401	Client non autenticato — verifica client_id/secret
403	Client autenticato ma senza permesso sull'operazione
429	Rate limit superato — attendi e riprova con backoff
500	Errore interno AS — retry con exponential backoff
503	AS temporaneamente non disponibile — retry

13.3 TLS — Il Prerequisito Assoluto

OAuth 2.0 richiede HTTPS per tutti gli endpoint. Senza TLS i token possono essere intercettati con un semplice man-in-the-middle attack. Non ci sono eccezioni in produzione.

■ TLS e header di sicurezza obbligatori

```
import requests, ssl

# SICURO: verifica certificato (default)
resp = requests.get('https://as.example.com/token', verify=True)

# Per CA self-signed in ambiente industriale (test/intranet):
resp = requests.get('https://local-as.factory.internal/token',
                    verify='/path/to/factory-ca.crt')

# PERICOLOSO — mai in produzione!
# resp = requests.get(url, verify=False) # Vulnerabile a MITM!

# Header di sicurezza obbligatori per il token endpoint (lato server):
from flask import make_response, jsonify
def secure_token_response(data):
    resp = make_response(jsonify(data))
    resp.headers['Cache-Control'] = 'no-store' # RFC 6749
    resp.headers['Pragma'] = 'no-cache' # RFC 6749
    resp.headers['X-Content-Type-Options'] = 'nosniff'
    resp.headers['X-Frame-Options'] = 'DENY'
    resp.headers['Strict-Transport-Security'] = 'max-age=31536000'
    return resp
```

Capitolo 14: Gestione degli Errori OAuth 2.0

OAuth 2.0 definisce un formato standard per gli errori. La corretta gestione è essenziale per un'implementazione robusta.

14.1 Errori del Token Endpoint

Codice	Causa	Azione
invalid_request	Parametri mancanti o duplicati	Correggi la richiesta
invalid_client	client_id/secret errati o sconosciuti	Verifica credenziali
invalid_grant	Code scaduto/usato o refresh non valido	Riavvia il flusso
unauthorized_client	Client non autorizzato per questo grant	Verifica registrazione
unsupported_grant_type	Grant type non supportato dall'AS	Usa grant diverso
invalid_scope	Scope non valido o non autorizzato	Riduci gli scope
server_error	Errore interno AS	Retry con backoff
temporarily_unavailable	AS sovraccarico	Exponential backoff

14.2 Client OAuth Robusto con Retry

■ Client OAuth con retry e gestione errori

```
import requests, time, logging

logger = logging.getLogger('oauth')
RETRYABLE = {'server_error', 'temporarily_unavailable'}

class RobustOAuthClient:
    def __init__(self, token_url, client_id, client_secret,
                 max_retries=3, base_delay=1.0):
        self.token_url = token_url
        self.client_id = client_id
        self.client_secret = client_secret
        self.max_retries = max_retries
        self.base_delay = base_delay

    def get_token(self, grant_type, **kwargs):
        data = {'grant_type': grant_type, 'client_id': self.client_id, **kwargs}
        for attempt in range(self.max_retries + 1):
            try:
                resp = requests.post(self.token_url,
                                     data=data, auth=(self.client_id, self.client_secret),
                                     timeout=10)
                if resp.status_code == 200:
                    return resp.json()
                err_data = resp.json()
                error_code = err_data.get('error', 'unknown')
                if error_code in RETRYABLE and attempt < self.max_retries:
                    wait = self.base_delay * (2 ** attempt) # Exponential backoff
                    logger.warning(f"Errore {error_code}, retry {attempt+1} tra {wait}s")
                    time.sleep(wait)
```

```
        continue
    # Errore non recuperabile
    desc = err_data.get('error_description', '')
    raise OAuthError(error_code, desc)
except requests.Timeout:
    if attempt < self.max_retries:
        time.sleep(self.base_delay * (2 ** attempt))
        continue
    raise

class OAuthError(Exception):
    def __init__(self, code, description=''):
        self.code = code
        self.description = description
        super().__init__(f"{code}: {description}")

# Utilizzo
client = RobustOAuthClient('https://as.example.com/token',
                           'my_app', 'my_secret')

try:
    token = client.get_token('client_credentials', scope='read write')
    print(f"Token OK, scade tra {token['expires_in']}s")
except OAuthError as e:
    if e.code == 'invalid_client':
        print("Credenziali non valide – verifica client_id e secret")
    elif e.code == 'invalid_scope':
        print("Scope non disponibile – verifica gli scope registrati")
    else:
        print(f"Errore OAuth: {e}")
```

14.3 Errori del Authorization Endpoint

Errore	Causa	Gestione
access_denied	Utente ha rifiutato il consenso	Mostra messaggio, non riprovare subito
invalid_scope	Scope non valido	Riduci scope e riproponi
login_required	Sessione scaduta (OIDC)	Forza ri-autenticazione
consent_required	Consenso necessario (OIDC)	Usa prompt=consent
server_error	Errore nell'AS	Retry, poi segnala all'utente
temporarily_unavailable	AS sovraccarico	Retry con backoff esponenziale

Capitolo 15: Testing di Applicazioni OAuth

Testare un'applicazione OAuth è complesso perché richiede l'interazione con un Authorization Server. Le strategie di mock permettono test veloci senza dipendenze esterne.

15.1 Mock dell'Authorization Server

■ Mock AS e unit test JWT

```
import pytest, jwt, time, secrets
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.backends import default_backend

# Chiavi RSA di test
PRIVATE_KEY = rsa.generate_private_key(65537, 2048, default_backend())
PUBLIC_KEY = PRIVATE_KEY.public_key()
TEST_ISSUER = 'https://test-as.example.com'
TEST_AUD = 'test_client'

def make_test_token(sub='user_01', email='test@example.com',
                   scope='read write', expired=False, minutes=60):
    now = int(time.time())
    exp = now - 10 if expired else now + minutes * 60
    payload = {'iss': TEST_ISSUER, 'sub': sub, 'aud': TEST_AUD,
              'exp': exp, 'iat': now, 'email': email, 'scope': scope,
              'jti': secrets.token_hex(8)}
    return jwt.encode(payload, PRIVATE_KEY, algorithm='RS256')

class MockAS:
    """Simula un Authorization Server per i test."""
    def __init__(self):
        self.issued = {}
        self.revoked = set()

    def issue(self, client_id, scope, sub='test_user'):
        tok = make_test_token(sub=sub, scope=scope)
        self.issued[tok] = {'client_id': client_id, 'scope': scope, 'sub': sub}
        return {'access_token': tok, 'token_type': 'Bearer', 'expires_in': 3600}

    def introspect(self, token):
        if token in self.revoked: return {'active': False}
        info = self.issued.get(token)
        return {'active': True, **info} if info else {'active': False}

    def revoke(self, token): self.revoked.add(token)

# Test
class TestOAuth:
    def setup_method(self):
        self.mock_as = MockAS()

    def test_valid_token(self):
        tok = make_test_token()
        data = jwt.decode(tok, PUBLIC_KEY, algorithms=['RS256'],
                          audience=TEST_AUD, issuer=TEST_ISSUER)
        assert data['sub'] == 'user_01'
```

```

def test_expired_token_rejected(self):
    tok = make_test_token(expired=True)
    with pytest.raises(jwt.ExpiredSignatureError):
        jwt.decode(tok, PUBLIC_KEY, algorithms=['RS256'],
                   audience=TEST_AUD, issuer=TEST_ISSUER)

def test_revoked_token(self):
    result = self.mock_as.issue('client', 'read')
    token = result['access_token']
    assert self.mock_as.introspect(token)['active'] == True
    self.mock_as.revoke(token)
    assert self.mock_as.introspect(token)['active'] == False

def test_scope_validation(self):
    tok = make_test_token(scope='read')
    data = jwt.decode(tok, PUBLIC_KEY, algorithms=['RS256'],
                     audience=TEST_AUD, issuer=TEST_ISSUER)
    assert 'write' not in data['scope'].split()

```

15.2 Integration Testing con unittest.mock

■ Integration test con mock HTTP

```

from unittest.mock import patch, MagicMock
import requests

```

```

class TestOAuthIntegration:

```

```

    @patch('requests.post')
    def test_client_credentials_call(self, mock_post):
        mock_post.return_value.status_code = 200
        mock_post.return_value.json.return_value = {
            'access_token': 'test_tok_xyz',
            'expires_in': 3600,
            'token_type': 'Bearer'
        }
        auth = ClientCredentialsAuth(
            'https://as.example.com/token', 'client_id', 'secret', 'read')
        token = auth.get_token()
        assert token == 'test_tok_xyz'
        mock_post.assert_called_once()
        call_args = mock_post.call_args
        assert call_args[1]['auth'] == ('client_id', 'secret')

    @patch('requests.post')
    def test_token_refresh_on_expiry(self, mock_post):
        mock_post.return_value.status_code = 200
        mock_post.return_value.json.return_value = {
            'access_token': 'new_tok', 'expires_in': 3600}
        mgr = TokenManager('https://as.example.com/token', 'c', 's')
        mgr.set_initial_tokens('old_tok', 'refresh_tok_xyz', expires_in=0)
        # expires_in=0 -> token subito scaduto -> deve refreshare
        new_tok = mgr.get_valid_token()
        assert new_tok == 'new_tok'
        assert mock_post.called

    @patch('requests.get')
    def test_api_call_with_bearer(self, mock_get):

```

```
mock_get.return_value.status_code = 200
mock_get.return_value.json.return_value = {'items': [1, 2, 3]}
token = 'valid_access_token'
resp = requests.get('https://api.example.com/data',
                    headers={'Authorization': f'Bearer {token}'})
assert resp.json()['items'] == [1, 2, 3]
headers = mock_get.call_args[1]['headers']
assert headers['Authorization'] == f'Bearer {token}'
```


Capitolo 16: Performance e Caching OAuth

In sistemi ad alto traffico, la validazione dei token può diventare un collo di bottiglia. Le chiavi JWKS cambiano raramente (ogni 24-48h): fare una chiamata HTTP per ogni validazione è inutile e lento.

16.1 Caching delle Chiavi JWKS

■ JWKS Cache thread-safe con pre-refresh

```
import time, threading, requests
from typing import Optional

class JWKSCache:
    """
    Cache thread-safe per chiavi pubbliche JWKS con TTL e pre-refresh.
    Essenziale per sistemi che validano migliaia di JWT al secondo.
    """
    def __init__(self, jwks_uri: str, ttl: int = 3600,
                  pre_refresh: int = 300):
        self.jwks_uri = jwks_uri
        self.ttl = ttl # Durata cache (default: 1 ora)
        self.pre_refresh = pre_refresh # Secondi prima della scadenza per refresh
        self._keys = {}
        self._raw = None
        self._fetched = 0.0
        self._lock = threading.RLock()
        self._refreshing = False

    def _expired(self) -> bool:
        return time.time() - self._fetched >= self.ttl

    def _stale(self) -> bool:
        age = time.time() - self._fetched
        return age >= (self.ttl - self.pre_refresh)

    def _fetch(self):
        from jwt.algorithms import RSAAlgorithm, ECAAlgorithm
        import json
        raw = requests.get(self.jwks_uri, timeout=5).json()
        keys = {}
        for k in raw.get('keys', []):
            kid = k.get('kid', 'default')
            kty = k.get('kty', 'RSA')
            keys[kid] = (RSAAlgorithm if kty == 'RSA' else ECAAlgorithm).from_jwk(js

        with self._lock:
            self._keys = keys
            self._raw = raw
            self._fetched = time.time()

    def _bg_refresh(self):
        if self._refreshing: return
        def _do():
            self._refreshing = True
            try: self._fetch()
            except Exception as e: print(f"JWKS refresh err: {e}")
            finally: self._refreshing = False
        threading.Thread(target=_do, daemon=True).start()
```

```

def get(self, kid: Optional[str] = None):
    with self._lock:
        if self._expired():
            self._fetch()          # Primo accesso o scaduto: sincrono
        elif self._stale():
            self._bg_refresh()     # Quasi scaduto: refresh in background

    if kid:
        if kid not in self._keys:
            self._fetch()          # kid sconosciuto (nuova chiave): forza fetch
        return self._keys.get(kid)
    return next(iter(self._keys.values()), None)

# Singleton globale
_jwks = JWKSCache('https://as.example.com/.well-known/jwks.json')

def fast_validate(token):
    import jwt
    hdr = jwt.get_unverified_header(token)
    key = _jwks.get(hdr.get('kid'))
    return jwt.decode(token, key, algorithms=['RS256'],
                      audience='my_api', issuer='https://as.example.com')

```

16.2 Benchmark: Con e Senza Cache

Scenario	Latenza per token	Throughput tipico
Senza cache (fetch JWKS ogni volta)	150-500 ms	2-7 validazioni/s
Con JWKSCache in memoria	0.5-2 ms	500-2000 val/s
Con Redis (valori precalcolati)	1-5 ms	200-1000 val/s
JWT self-contained verificato in-process	0.1-0.5 ms	2000-10000 val/s

16.3 Token Caching con Redis

■ Token validation caching con Redis

```

# pip install redis
import redis, json, hashlib, time

class RedisTokenCache:
    """
    Cache distribuita per validazioni JWT in ambienti multi-server.
    Evita di rivalidare crittograficamente lo stesso token ad ogni richiesta.
    """
    def __init__(self, redis_url='redis://localhost:6379/0'):
        self.r = redis.from_url(redis_url)
        self.prefix = 'oauth_tok:'

    def _key(self, token: str) -> str:
        return self.prefix + hashlib.sha256(token.encode()).hexdigest()[:16]

    def get(self, token: str):
        raw = self.r.get(self._key(token))
        return json.loads(raw) if raw else None

    def set(self, token: str, payload: dict, exp: int):

```

```
        ttl = max(1, int(exp - time.time()))
        self.r.setex(self._key(token), ttl, json.dumps(payload))

    def invalidate(self, token: str):
        self.r.delete(self._key(token)) # Utile dopo revoca

_cache = RedisTokenCache()

def cached_validate(token):
    # 1. Controlla cache Redis (O(1), ~1ms)
    cached = _cache.get(token)
    if cached:
        return cached # Cache hit – nessuna crittografia

    # 2. Valida JWT (crittografico, ~1-2ms con chiavi in memoria)
    payload = fast_validate(token)

    # 3. Salva in cache per le prossime richieste
    _cache.set(token, payload, payload['exp'])
    return payload
```

Capitolo 17: Single Sign-On con OpenID Connect

Single Sign-On permette agli utenti di autenticarsi una volta e accedere a più applicazioni senza reinserire le credenziali. OIDC/OAuth 2.0 è il protocollo standard per implementare SSO.

17.1 Come Funziona il SSO con OIDC

Passo	Evento
1	Utente visita App A
2	App A reindirige al Keycloak (primo login: inserisce credenziali)
3	Keycloak crea sessione SSO, reindirige ad App A con code
4	App A scambia code per token, utente autenticato in App A
...	Utente usa App A normalmente
5	Utente visita App B (stessa sessione browser)
6	App B reindirige al Keycloak
7	Keycloak riconosce sessione SSO esistente — NON chiede login
8	Keycloak reindirige ad App B con code
9	App B scambia code per token — utente autenticato senza digitare nulla

✓ Il SSO funziona grazie ai cookie di sessione del browser con il dominio dell'AS. Quando App B reindirige al Keycloak, il browser invia automaticamente il cookie della sessione SSO precedente.

17.2 SSO Logout — Front-Channel e Back-Channel

Il logout SSO è più complesso del login. Esistono due meccanismi: Front-Channel (basato su browser, meno affidabile) e Back-Channel (server-to-server, RFC OIDC Back-Channel Logout, più robusto).

■ SSO logout front-channel e back-channel

```
# Logout front-channel: reindirizza al Keycloak per invalidare sessione SSO
from flask import session, redirect

@app.route('/logout')
def logout():
    id_token = session.get('id_token', '')
    session.clear() # Pulisce sessione locale
    # Keycloak invalida la sessione SSO e notifica tutte le app (se configurate)
    return redirect(
        'https://keycloak.factory.com/realms/industrial/protocol/'
        f'openid-connect/logout?id_token_hint={id_token}'
        '&post_logout_redirect_uri=https://myapp.factory.com'
    )

# Back-Channel logout: Keycloak chiama questo endpoint su ogni app
@app.route('/backchannel-logout', methods=['POST'])
```

```
def backchannel_logout():
    """Keycloak chiama questo endpoint quando un utente fa logout da qualsiasi app."""
    logout_token = request.form.get('logout_token')
    if not logout_token:
        return 'Missing logout_token', 400
    try:
        # Verifica il logout_token (JWT speciale con claim 'events')
        import jwt as pyjwt
        jwks = requests.get(JWKS_URI).json()
        claims = pyjwt.decode(logout_token, jwks, algorithms=['RS256'],
                               audience=CLIENT_ID, options={'verify_exp': False})
        # Verifica che sia un logout token, non un ID token normale
        events = claims.get('events', {})
        if 'http://schemas.openid.net/event/backchannel-logout' not in events:
            return 'Not a logout token', 400
        # Invalida la sessione dell'utente in Redis/DB
        sub = claims.get('sub')
        sid = claims.get('sid')
        invalidate_sessions(sub, sid) # Implementa in base al tuo storage
        return '', 200
    except pyjwt.InvalidTokenError as e:
        return f'Invalid token: {e}', 400

def invalidate_sessions(sub, sid):
    # Con Redis: cancella tutte le sessioni dell'utente
    import redis
    r = redis.Redis()
    for key in r.scan_iter(f'session:{sub}:*'):
        r.delete(key)
```

Capitolo 18: Keycloak — Authorization Server Open Source

Keycloak è l'Authorization Server open source più usato per deployment on-premise o cloud privato. Ideale per ambienti industriali dove non si vuole dipendere da provider cloud esterni.

18.1 Avvio Rapido con Docker

■ Keycloak avvio Docker (dev + prod)

```
# Avvio Keycloak con Docker (sviluppo)
docker run -p 8080:8080 \
  -e KEYCLOAK_ADMIN=admin \
  -e KEYCLOAK_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:latest start-dev

# Produzione: con PostgreSQL e TLS
docker run -p 8443:8443 \
  -e KEYCLOAK_ADMIN=admin \
  -e KEYCLOAK_ADMIN_PASSWORD=${KC_ADMIN_SECRET} \
  -e KC_DB=postgres \
  -e KC_DB_URL=jdbc:postgresql://db:5432/keycloak \
  -e KC_DB_USERNAME=keycloak \
  -e KC_DB_PASSWORD=${DB_SECRET} \
  -e KC_HOSTNAME=auth.factory.com \
  quay.io/keycloak/keycloak:latest start
```

18.2 Configurazione via python-keycloak

■ Configurazione Keycloak via API Python

```
# pip install python-keycloak
from keycloak import KeycloakAdmin

admin = KeycloakAdmin(
    server_url = 'http://localhost:8080/',
    username   = 'admin', password='admin',
    realm_name = 'master', verify=True)

# Crea realm per impianto industriale
admin.create_realm({
    'realm': 'impianto-acque', 'enabled': True,
    'displayName': 'Impianto Trattamento Acque',
    'sslRequired': 'external',
    'accessTokenLifespan': 300, # 5 minuti
    'refreshTokenMaxReuse': 0, # Rotation obbligatoria
    'revokeRefreshToken': True,
})

# Crea client HMI
admin.realm_name = 'impianto-acque'
admin.create_client({
    'clientId': 'hmi_dashboard',
    'enabled': True,
    'protocol': 'openid-connect',
    'publicClient': False,
```

```

'secret':      'hmi_secret_secure_123',
'standardFlowEnabled': True,    # Authorization Code
'serviceAccountsEnabled': True, # Client Credentials
'redirectUri': ['https://hmi.factory.local/callback'],
'defaultClientScopes': ['openid', 'profile', 'email'],
})

# Crea ruoli
for role in ['operatore', 'supervisore', 'admin_impianto']:
    admin.create_realm_role({'name': role})

# Crea utente operatore
user_id = admin.create_user({
    'username': 'mario.rossi',
    'email':    'mario.rossi@factory.com',
    'enabled':  True,
    'firstName': 'Mario', 'lastName': 'Rossi',
    'credentials': [{ 'type': 'password', 'value': 'TempPass123!',
                       'temporary': True}]
})
admin.assign_realm_roles(user_id, [{'name': 'operatore'}])
print(f"Keycloak configurato! User ID: {user_id}")

```

18.3 Login Operatori HMI con Keycloak

■ Login HMI con Keycloak

```

from keycloak import KeycloakOpenID

kc = KeycloakOpenID(
    server_url      = 'https://auth.factory.com/',
    realm_name      = 'impianto-acque',
    client_id       = 'hmi_dashboard',
    client_secret_key = 'hmi_secret_secure_123'
)

def login_operatore(username, password):
    """Autenticazione HMI con Keycloak."""
    try:
        token = kc.token(username, password)
        info = kc.introspect(token['access_token'])
        if not info['active']:
            return None, 'Token non attivo'
        return {
            'access_token': token['access_token'],
            'refresh_token': token['refresh_token'],
            'user':        info.get('preferred_username'),
            'roles':        info.get('realm_access', {}).get('roles', []),
            'expires_in':   token.get('expires_in'),
        }, None
    except Exception as e:
        return None, str(e)

def can(token_str, role):
    """Controlla se l'operatore ha il ruolo necessario."""
    info = kc.introspect(token_str)
    return role in info.get('realm_access', {}).get('roles', [])

# Utilizzo nell'HMI

```

```
session, err = login_operatore('mario.rossi', 'password')
if session:
    print(f"Login OK: {session['user']} – ruoli: {session['roles']}")
    if can(session['access_token'], 'supervisore'):
        print("Accesso supervisore concesso")
    else:
        print("Accesso supervisore negato")
else:
    print(f"Login fallito: {err}")
```


Capitolo 19: OAuth 2.0 nei Microservizi

I microservizi presentano sfide uniche: ogni servizio è un Resource Server, i token devono propagarsi tra servizi, e le audience devono essere configurate correttamente.

19.1 Pattern di Propagazione del Token

Pattern	Descrizione	Pro/Contro
Token pass-through	Il gateway passa il token originale	Semplice / token valido per tutti
Token exchange	Servizio scambia per un token audience-specifico (RFC 8414)	Simple / Overhead
Service token	Ogni servizio usa Client Credentials	Isolato / nessun contesto utente
JWT self-contained	JWT con tutti i claims necessari	Stateless / token grandi

19.2 API Gateway con Validazione JWT Centralizzata

■ API Gateway con middleware JWT

```
# Middleware OAuth per API Gateway
from flask import Flask, request, jsonify, g
from functools import wraps, lru_cache
import jwt, requests, time

app = Flask(__name__)

@lru_cache(maxsize=1)
def _get_jwks():
    """Cache JWKS – aggiornata al riavvio. In produzione usa JWKSCache sopra."""
    return requests.get('https://as.factory.com/.well-known/jwks.json').json()

def require_auth(*scopes):
    """Decorator: valida JWT e controlla scope opzionali."""
    def decorator(f):
        @wraps(f)
        def wrapped(*args, **kwargs):
            auth = request.headers.get('Authorization', '')
            if not auth.startswith('Bearer '):
                return jsonify({'error': 'missing_token'}), 401
            token = auth[7:]
            try:
                payload = jwt.decode(token, _get_jwks(),
                                     algorithms=['RS256'], audience='factory_api',
                                     issuer='https://as.factory.com')
            except jwt.ExpiredSignatureError:
                return jsonify({'error': 'token_expired'}), 401
            except jwt.InvalidTokenError as e:
                return jsonify({'error': str(e)}), 401

            if scopes:
                token_scopes = set(payload.get('scope', '').split())
                missing = set(scopes) - token_scopes
                if missing:
                    return jsonify({'error': 'insufficient_scope',
                                    'required': list(scopes)}), 403
```

```
        g.user = {'id': payload['sub'],
                  'email': payload.get('email'),
                  'roles': payload.get('roles', [])}
        # Propaga all'header per i microservizi interni
        request.environ['HTTP_X_USER_ID'] = payload['sub']
        request.environ['HTTP_X_USER_ROLES'] = ','.join(payload.get('roles', []))
        return f(*args, **kwargs)
    return wrapped
return decorator

# Endpoint protetti
@app.route('/api/v1/telemetry')
@require_auth('telemetry:read')
def get_telemetry():
    return jsonify({'user': g.user['id'], 'data': [1, 2, 3]})

@app.route('/api/v1/control')
@require_auth('telemetry:read', 'control:write')
def send_control():
    return jsonify({'status': 'ok', 'user': g.user['id']})
```

Capitolo 20: Argomenti Avanzati — PAR, DPOP, mTLS

OAuth 2.0 continua a evolversi con estensioni che aumentano sicurezza e flessibilità. Questa sezione introduce le estensioni più importanti degli ultimi anni.

20.1 PAR — Pushed Authorization Requests (RFC 9126)

PAR permette al client di inviare i parametri di autorizzazione direttamente all'AS via POST (server-to-server) invece che nell'URL. L'AS restituisce un `request_uri` opaco da usare nel redirect. Vantaggi: parametri non visibili nel browser history, supporta richieste grandi.

■ PAR — Pushed Authorization Requests

```
import requests, secrets, urllib.parse

def par_authorization_flow(client_id, client_secret, redirect_uri,
                           scope, par_endpoint, auth_endpoint):
    """
    Pushed Authorization Request – RFC 9126.
    I parametri vengono inviati al server prima del redirect.
    """
    state = secrets.token_urlsafe(32)
    # Step 1: invia i parametri via POST all'AS (server-to-server)
    par_resp = requests.post(par_endpoint, data={
        'response_type': 'code',
        'client_id':     client_id,
        'redirect_uri':  redirect_uri,
        'scope':         scope,
        'state':         state,
    }, auth=(client_id, client_secret))

    par_resp.raise_for_status()
    data = par_resp.json()
    request_uri = data['request_uri']    # Es. urn:ietf:params:oauth:request_uri:6ue3...
    expires_in  = data['expires_in']    # Tipicamente 60 secondi

    # Step 2: reindirizza usando solo il request_uri (opaco, non contiene dati sensibili)
    auth_url = f"{auth_endpoint}?" + urllib.parse.urlencode({
        'client_id':     client_id,
        'request_uri': request_uri,    # Tutto il payload e' sull'AS
    })
    return auth_url, state, expires_in

auth_url, state, exp = par_authorization_flow(
    client_id='my_app', client_secret='secret',
    redirect_uri='https://myapp.com/callback',
    scope='openid email profile',
    par_endpoint='https://as.example.com/par',
    auth_endpoint='https://as.example.com/authorize'
)
print(f"Redirect (URL breve, senza parametri sensibili): {auth_url}")
print(f"Valido per {exp} secondi")
```

20.2 DPOP — Demonstration of Proof of Possession (RFC 9449)

DPoP lega il token a una coppia di chiavi crittografiche del client. Un token DPoP non può essere usato da chi lo ruba: serve anche la chiave privata. Risolve il problema fondamentale dei Bearer token (usabili da chiunque li possiede).

■ DPoP — Proof of Possession

```
# DPoP – Demonstration of Proof of Possession
# Il client dimostra di possedere la chiave privata ad ogni richiesta
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.backends import default_backend
import jwt, time, uuid, json, base64

# Genera coppia di chiavi EC per DPoP (una volta, memorizzata sicuramente)
dpop_private_key = ec.generate_private_key(ec.SECP256R1(), default_backend())
dpop_public_key = dpop_private_key.public_key()

def make_dpop_proof(http_method: str, http_uri: str, ath: str = None) -> str:
    """
    Crea un DPoP proof JWT per una richiesta specifica.
    Ogni prova e' monouso (jti univoco) e legata al metodo+URI.
    """
    from cryptography.hazmat.primitives.serialization import Encoding, PublicFormat
    pub_jwk = {
        'kty': 'EC', 'crv': 'P-256',
        'x': base64.urlsafe_b64encode(
            dpop_public_key.public_numbers().x.to_bytes(32, 'big')).rstrip(b'=').decode(),
        'y': base64.urlsafe_b64encode(
            dpop_public_key.public_numbers().y.to_bytes(32, 'big')).rstrip(b'=').decode(),
    }
    header = {'typ': 'dpop+jwt', 'alg': 'ES256', 'jwk': pub_jwk}
    payload = {'jti': str(uuid.uuid4()), # Monouso
               'htm': http_method, # HTTP method (GET, POST, ...)
               'htu': http_uri, # HTTP URI
               'iat': int(time.time())}
    if ath:
        # Bind al token (SHA-256 base64url dell'access token)
        payload['ath'] = ath
    return jwt.encode(payload, dpop_private_key, algorithm='ES256',
                      headers=header)

# Utilizzo: ogni richiesta API include il DPoP proof
def dpop_request(access_token, method, url):
    import hashlib
    ath = base64.urlsafe_b64encode(
        hashlib.sha256(access_token.encode()).digest()).rstrip(b'=').decode()
    proof = make_dpop_proof(method.upper(), url, ath=ath)
    return requests.request(method, url, headers={
        'Authorization': f'DPoP {access_token}', # DPoP invece di Bearer!
        'DPoP': proof # Prova di possesso della chiave
    })
```

Capitolo 21: Conformita' e Standard Industriali

In ambienti industriali e regolamentati, la conformità agli standard è spesso obbligatoria. Questa sezione copre come OAuth 2.0 si integra con i principali framework di sicurezza.

21.1 IEC 62443 e OAuth 2.0

IEC 62443 è lo standard per la cybersecurity di sistemi di controllo industriali (ICS/OT). OAuth 2.0 può essere usato per implementare i requisiti di autenticazione e autorizzazione a vari Security Level (SL).

IEC 62443 SL	Requisiti auth	Implementazione OAuth
SL 1	Autenticazione di base	Client Credentials + TLS
SL 2	Autenticazione multi-fattore (MFA)	OIDC con MFA su Keycloak
SL 3	Autenticazione forte + audit trail	PKCE + JWT + log centralizzato
SL 4	Protezione massima + hardware token	mTLS + DPoP + HSM per chiavi

21.2 Audit Trail con OAuth

■ Audit trail OAuth per IEC 62443

```
# Audit trail per ogni accesso OAuth in ambiente industriale
import time, json, logging
from flask import request, g

audit_logger = logging.getLogger('oauth_audit')
audit_logger.setLevel(logging.INFO)

def log_oauth_event(event_type: str, user_id: str, resource: str,
                    scope: str, result: str, details: dict = None):
    """
    Registra ogni evento di accesso OAuth per audit IEC 62443.
    Include: chi, cosa, quando, da dove, con quali permessi, esito.
    """
    event = {
        'timestamp': time.strftime('%Y-%m-%dT%H:%M:%SZ', time.gmtime()),
        'event_type': event_type,          # LOGIN, API_CALL, LOGOUT, TOKEN_REFRESH
        'user_id': user_id,
        'client_id': getattr(g, 'client_id', 'unknown'),
        'resource': resource,
        'scope': scope,
        'result': result,                  # SUCCESS, DENIED, ERROR
        'source_ip': request.remote_addr,
        'user_agent': request.headers.get('User-Agent', ''),
        'token_jti': getattr(g, 'token_jti', 'unknown'), # Per correlare
    }
    if details:
        event['details'] = details
    # In produzione: invia a SIEM (Splunk, Elastic, ecc.)
    audit_logger.info(json.dumps(event))

# Utilizzo nei decorator
def require_auth_with_audit(resource: str, *scopes):
    def decorator(f):
```

```

@wraps(f)
def wrapped(*args, **kwargs):
    token = extract_token() # Estrai il Bearer token
    try:
        payload = validate_jwt(token)
        g.user_id = payload['sub']
        g.client_id = payload.get('client_id')
        g.token_jti = payload.get('jti')
        log_oauth_event('API_CALL', g.user_id, resource,
                        payload.get('scope', ''), 'SUCCESS')
        return f(*args, **kwargs)
    except Exception as e:
        log_oauth_event('API_CALL', 'unknown', resource, '',
                        'DENIED', {'error': str(e)})
        return jsonify({'error': 'unauthorized'}), 401
    return wrapped
return decorator

```

21.3 Rotazione Automatica dei Segreti

■ Rotazione segreti OAuth con HashiCorp Vault

Rotazione automatica client_secret con HashiCorp Vault

import hvac, os

```

vault_client = hvac.Client(url=os.environ['VAULT_ADDR'],
                           token=os.environ['VAULT_TOKEN'])

```

```

def get_current_secret(client_id: str) -> str:
    """Leggi il client_secret corrente da Vault (ruotato ogni 30 giorni)."""
    path = f'oauth/clients/{client_id}'
    secret = vault_client.secrets.kv.v2.read_secret_version(path=path)
    return secret['data']['data']['client_secret']

```

```

def rotate_client_secret(client_id: str, new_secret: str):
    """
    Ruota il client_secret in Vault e aggiorna il provider OAuth.
    In produzione: usa un secret manager (Vault, AWS Secrets Manager, Azure KV).
    """
    # Aggiorna su Vault
    vault_client.secrets.kv.v2.create_or_update_secret(
        path=f'oauth/clients/{client_id}',
        secret={'client_secret': new_secret,
               'rotated_at': time.strftime('%Y-%m-%dT%H:%M:%SZ', time.gmtime())}
    )
    # Aggiorna su Keycloak (via API admin)
    from keycloak import KeycloakAdmin
    admin = KeycloakAdmin(server_url='https://auth.factory.com/',
                          username='admin', password=get_current_secret('admin'))
    client_id_kc = admin.get_client_id(client_id)
    admin.set_client_secret(client_id_kc, new_secret)
    print(f"Secret ruotato per client {client_id}")

```

```

# Schedulazione rotazione (es. con APScheduler ogni 30 giorni)
# from apscheduler.schedulers.background import BackgroundScheduler
# scheduler = BackgroundScheduler()
# scheduler.add_job(lambda: rotate_client_secret('hmi_dashboard', secrets.token_hex(32)),
#                   'interval', days=30)

```

```
# scheduler.start()
```

Capitolo 22: OAuth 2.0 vs Alternative — SAML, API Key, Basic Auth

OAuth 2.0 non è l'unica soluzione per autenticazione e autorizzazione. Capire quando usarlo e quando preferire alternative è fondamentale.

Meccanismo	Tipo	Pro	Contro	Usa quando
OAuth 2.0	Framework	Standard aperto, delegazione, complessità elevata	Complessità, attacchi	Integrazione terze parti, API pubblica, Mobile
SAML 2.0	Protocollo	SSO enterprise, maturo, AD FS	SAML verboso, complesso	SSO enterprise Microsoft, legacy
API Key	Statico	Semplicissimo	Nessuna scadenza, no delega	Testa, API interne, semplice
Basic Auth	HTTP	Universale, browser nativo	Password in ogni richiesta (legato, HTTPS)	Legacy, HTTP tools
JWT bearer	Token	Stateless, self-contained	Revoca difficile	Microservizi interni, API senza OAuth A
mTLS	Cert	Massima sicurezza, M2M	Gestione certificati complesso	Industriale, FIPS compliance

22.1 OAuth 2.0 vs SAML — Quando Scegliere Cosa

Aspetto	OAuth 2.0 + OIDC	SAML 2.0
Protocollo	REST/JSON	SOAP/XML
SSO	Sì (via OIDC/sessioni AS)	Sì (core feature)
Mobile	Nativo (PKCE, Device Flow)	Difficile
API auth	Sì (access token)	No (non progettato per API)
Legacy AD	Via Azure AD/ADFS bridge	Nativo (ADFS)
Complessità	Media	Alta (XML, firme, metadati)
Adozione nuovi sistemi	Molto alta	In declino per nuovi sistemi

✓ Per nuovi sistemi usa sempre OAuth 2.0 + OIDC. SAML è ancora necessario per integrarsi con sistemi legacy Microsoft (ADFS) o piattaforme enterprise esistenti.

22.2 API Key — Quando Ha Ancora Senso

Le API Key hanno ancora un ruolo in scenari specifici: sono semplicissime da implementare e possono essere appropriate per API interne o per sviluppatori che testano le API.

■ API Key vs OAuth 2.0 Client Credentials

Confronto: API Key vs OAuth 2.0 Client Credentials

[illegible]

```
API_KEY = 'sk-abc123xyz'    # Statica, nessuna scadenza, nessuna revoca fine-grained
```

```
resp = requests.get('https://api.example.com/data',
```


Capitolo 23: Casi Studio Completi

Questa sezione presenta casi studio end-to-end che mostrano come OAuth 2.0 viene applicato in scenari reali.

23.1 Caso Studio: Dashboard SCADA con SSO

Scenario: un'azienda industriale vuole proteggere la dashboard SCADA di un impianto di trattamento acque. Gli operatori devono autenticarsi con le credenziali aziendali (Active Directory via Keycloak). Esistono tre ruoli: operatore (solo lettura), supervisore (lettura + comandi), admin (tutto).

■ Dashboard SCADA Flask completa con Keycloak e ruoli

```
# Sistema SCADA completo con OAuth Keycloak + ruoli + audit
import os, time, secrets, hashlib, base64, urllib.parse
import requests
from flask import Flask, redirect, request, session, jsonify
from functools import wraps

app = Flask(__name__)
app.secret_key = secrets.token_hex(32)

KC_BASE = os.environ.get('KEYCLOAK_URL', 'https://auth.factory.com')
REALM = 'impianto-acque'
CLIENT_ID = 'scada_dashboard'
CLIENT_SECRET = os.environ['SCADA_SECRET']
REDIRECT = 'https://scada.factory.local/callback'

OIDC = requests.get(
    f'{KC_BASE}/realms/{REALM}/.well-known/openid-configuration').json()

def require_role(*roles):
    """Decorator: richiede login + almeno uno dei ruoli specificati."""
    def deco(f):
        @wraps(f)
        def wrapped(*args, **kwargs):
            if 'access_token' not in session:
                session['next'] = request.url
                return redirect('/login')
            if session.get('token_expiry', 0) < time.time():
                try: _do_refresh()
                except: return redirect('/login')
            user_roles = session.get('roles', [])
            if roles and not any(r in user_roles for r in roles):
                return jsonify({'error': 'Ruolo insufficiente'}), 403
            return f(*args, **kwargs)
        return wrapped
    return deco

def _do_refresh():
    resp = requests.post(OIDC['token_endpoint'], data={
        'grant_type': 'refresh_token',
        'refresh_token': session['refresh_token'],
        'client_id': CLIENT_ID, 'client_secret': CLIENT_SECRET,
    }).json()
    _save_session(resp)
```

```

def _save_session(tokens):
    import jwt as pyjwt
    session['access_token'] = tokens['access_token']
    session['refresh_token'] = tokens.get('refresh_token')
    session['token_expiry'] = time.time() + tokens.get('expires_in', 300) - 30
    try:
        p = pyjwt.decode(tokens['access_token'], options={'verify_signature': False})
        session['user'] = p.get('preferred_username')
        session['email'] = p.get('email')
        session['roles'] = p.get('realm_access', {}).get('roles', [])
    except: pass

@app.route('/login')
def login():
    v = secrets.token_urlsafe(32)
    c = base64.urlsafe_b64encode(hashlib.sha256(v.encode()).digest()).rstrip(b'=').decode()
    s_val = secrets.token_urlsafe(16)
    session.update({'pkce_v': v, 'state': s_val})
    p = {'response_type': 'code', 'client_id': CLIENT_ID,
        'redirect_uri': REDIRECT, 'scope': 'openid email profile roles',
        'state': s_val, 'code_challenge': c, 'code_challenge_method': 'S256'}
    return redirect(f"{OIDC['authorization_endpoint']}?{urllib.parse.urlencode(p)}")

@app.route('/callback')
def callback():
    if request.args.get('state') != session.pop('state', None):
        return 'CSRF', 403
    tokens = requests.post(OIDC['token_endpoint'], data={
        'grant_type': 'authorization_code', 'code': request.args['code'],
        'redirect_uri': REDIRECT, 'client_id': CLIENT_ID,
        'client_secret': CLIENT_SECRET, 'code_verifier': session.pop('pkce_v', '')},
    ).json()
    _save_session(tokens)
    return redirect(session.pop('next', '/dashboard'))

@app.route('/dashboard')
@require_role('operatore', 'supervisore', 'admin_impianto')
def dashboard():
    return jsonify({'user': session['user'], 'roles': session['roles'],
        'message': 'Dashboard SCADA – accesso autorizzato'})

@app.route('/api/control', methods=['POST'])
@require_role('supervisore', 'admin_impianto')
def control():
    cmd = request.json.get('command')
    # Audit del comando
    log_oauth_event('CONTROL', session['user'], '/api/control',
        'control:write', 'SUCCESS', {'cmd': cmd})
    return jsonify({'status': 'ok', 'command': cmd, 'user': session['user']})

@app.route('/logout')
def logout():
    session.clear()
    return redirect(f"{OIDC['end_session_endpoint']}?"
        f"post_logout_redirect_uri=https://scada.factory.local")

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=443, ssl_context='adhoc')

```

23.2 Caso Studio: CI/CD Pipeline con GitHub OAuth

■ OIDC Federation — GitHub Actions senza secret

```
# GitHub Actions: accesso OAuth per deploy automatico
# Usa OIDC federation – GitHub Actions ottiene token da GitHub,
# che viene accettato da AWS/GCP/Azure senza client_secret hardcoded

# .github/workflows/deploy.yml
# name: Deploy to Factory Cloud
# on: [push]
# jobs:
#   deploy:
#     permissions:
#       id-token: write    # Permette di richiedere il token OIDC
#       contents: read
#     steps:
#       - uses: aws-actions/configure-aws-credentials@v4
#         with:
#           role-to-assume: arn:aws:iam::123456789:role/GitHubActions
#           aws-region: eu-west-1
#           # AWS verifica il JWT GitHub tramite OIDC federation – nessun secret!

# Codice Python per verificare il JWT GitHub in un servizio custom:
import jwt, requests

GITHUB_JWKS = 'https://token.actions.githubusercontent.com/.well-known/jwks.json'

def verify_github_actions_token(token, expected_repo):
    """Verifica un JWT emesso da GitHub Actions."""
    jwks = requests.get(GITHUB_JWKS).json()
    try:
        claims = jwt.decode(token, jwks, algorithms=['RS256'],
                             audience='https://factory-api.example.com',
                             issuer='https://token.actions.githubusercontent.com')
        # Verifica il repo (claim specifico GitHub)
        repo = claims.get('repository')
        if repo != expected_repo:
            raise ValueError(f"Repo non autorizzato: {repo}")
        return claims
    except jwt.InvalidTokenError as e:
        raise ValueError(f"Token GitHub non valido: {e}")

# Un servizio di deploy verifica il token prima di eseguire
# qualsiasi operazione sul cluster o sull'infrastruttura
token_from_ci = request.headers.get('Authorization', '').replace('Bearer ', '')
claims = verify_github_actions_token(token_from_ci, 'myorg/factory-app')
print(f"Deploy autorizzato da: {claims.get('actor')} ({claims.get('ref')})")
```

Approfondimento: Il Contesto Normativo Europeo di OAuth 2.0

Il Regolamento Generale sulla Protezione dei Dati (GDPR, 2018) ha avuto un impatto significativo su come OAuth 2.0 viene implementato in Europa. Il GDPR richiede che il consenso sia granulare, revocabile e basato su una base legale. OAuth 2.0, con il suo meccanismo di scope e la schermata di consenso, si allinea naturalmente a questi requisiti, a patto di implementarlo correttamente.

Il principio di minimizzazione dei dati del GDPR (Art. 5) corrisponde perfettamente al principio del minimo privilegio di OAuth 2.0: chiedere solo gli scope strettamente necessari alla funzionalità dell'applicazione. Richiedere scope eccessivi non solo è una cattiva pratica di sicurezza, ma può violare il GDPR.

Il Regolamento eIDAS (electronic IDentification, Authentication and trust Services) definisce i livelli di garanzia dell'identità elettronica in Europa (basso, sostanziale, elevato). OpenID Connect e OAuth 2.0 sono i protocolli tecnici preferiti per implementare sistemi di identità conformi a eIDAS. La Commissione Europea ha scelto OIDC per il portafoglio di identità digitale europeo (EU Digital Identity Wallet, 2026).

La Direttiva NIS2 (Network and Information Systems, 2022) impone requisiti di sicurezza più stringenti per operatori di servizi essenziali, inclusi impianti idrici, energetici e manifattura critica. L'autenticazione forte con MFA — implementabile tramite OIDC con Keycloak e provider MFA come TOTP o WebAuthn — diventa obbligatoria per questi settori.

La Direttiva PSD2 (Payment Services Directive 2) per i servizi finanziari richiede Strong Customer Authentication (SCA) e Open Banking. OAuth 2.0 è il cuore tecnico di tutte le implementazioni Open Banking europee, con profili specifici come UK Open Banking e Berlin Group NEXTGEN PSD2.

- GDPR Art. 5: minimizzazione dei dati — richiedere solo scope necessari
- GDPR Art. 7: revoca del consenso — il meccanismo di revoca OAuth è il mezzo tecnico
- eIDAS LoA: usare MFA per livelli 'sostanziale' ed 'elevato'
- NIS2: autenticazione forte obbligatoria per infrastrutture critiche
- PSD2: OAuth 2.0 come base tecnica dell'Open Banking europeo

Sicurezza by Design — Principi Fondamentali

Il principio di 'Sicurezza by Design' (Security by Design) richiede che la sicurezza sia incorporata nel sistema fin dall'inizio, non aggiunta come strato successivo. In OAuth 2.0, questo significa scegliere il grant type corretto dal primo giorno, implementare PKCE anche quando non sembra necessario, e progettare la gestione dei token come parte integrante dell'architettura.

Il Defense in Depth (difesa in profondità) applica multiple misure di sicurezza indipendenti. In un sistema OAuth: TLS per il trasporto, PKCE per prevenire l'intercettazione del code, state per il CSRF, validazione strict del redirect_uri, scope minimali, token con vita breve, refresh token rotation, revoca al logout. Ognuno di questi strati è indipendente: se uno fallisce, gli altri proteggono ancora.

Il principio del Least Privilege (minimo privilegio) è direttamente implementato dagli scope OAuth. Un'app che ha solo bisogno di leggere il profilo utente non deve richiedere lo scope di lettura email. Un microservizio che invia dati non deve avere scope per leggerli. Questo limita il danno in caso di compromissione di un token.

Il principio di Fail Secure richiede che in caso di errore il sistema vada verso lo stato più sicuro, non verso quello più permissivo. Se la validazione del JWT fallisce per un errore tecnico (rete, timeout), il

sistema deve rifiutare la richiesta, non accettarla. Le eccezioni non gestite non devono mai risultare in accesso non autorizzato.

Il principio di Complete Mediation richiede che ogni accesso a ogni risorsa protetta venga verificato ogni volta. Non si deve mai assumere che se una richiesta precedente era autorizzata, le successive lo siano automaticamente. Il caching della validazione JWT è accettabile, ma con TTL breve e invalidazione al logout.

- Security by Design: scegli il grant type corretto dall'inizio
- Defense in Depth: TLS + PKCE + state + redirect_uri + scope + token breve
- Least Privilege: scope minimi per ogni applicazione
- Fail Secure: errore di validazione = rifiuto, mai accesso concesso
- Complete Mediation: ogni accesso verificato, nessuna assunzione

Evoluzione Storica: Da OAuth 1.0 a OAuth 2.1

La transizione da OAuth 1.0 a OAuth 2.0 fu controversa. Eran Hammer-Lahav, l'editor principale del RFC 5849 (OAuth 1.0) e inizialmente anche di OAuth 2.0, si ritirò dal working group nel 2012 con un post famoso: 'OAuth 2.0 and the Road to Hell'. Criticava il fatto che OAuth 2.0, pur essendo più semplice per i sviluppatori di client, era meno sicuro di OAuth 1.0 perché richiedeva HTTPS obbligatorio ma non lo garantiva.

Le preoccupazioni di Hammer-Lahav si sono rivelate in parte fondate: molte implementazioni iniziali di OAuth 2.0 avevano vulnerabilità proprio nei punti che lui aveva evidenziato — redirect_uri validation insufficiente, state parameter mancante, Implicit Flow insicuro. Tuttavia, il successo commerciale di OAuth 2.0 è stato straordinario: Google, Facebook, GitHub, Twitter, Microsoft l'hanno adottato in pochi anni.

OAuth 2.0 ha anche abilitato l'Open Banking, l'ecosistema delle API cloud, e tutta la moderna architettura a microservizi. La semplicità — anche se a costo di meno garanzie di sicurezza out-of-the-box — si è rivelata la scelta giusta per l'adozione di massa. Le vulnerabilità sono state progressivamente mitigate con estensioni (PKCE nel 2015, Token Exchange nel 2019, PAR nel 2021).

OAuth 2.1 (draft dal 2020, ancora in corso nel 2025) non aggiunge nuove funzionalità: consolida le best practice apprese in 10+ anni di deployment. La filosofia è 'rimuovere ciò che è pericoloso, rendere obbligatorio ciò che è sicuro'. PKCE obbligatorio, Implicit Flow rimosso, ROPC rimosso, redirect_uri match esatto. Per le nuove implementazioni, seguire OAuth 2.1 fin dall'inizio è la scelta corretta.

- 2007: OAuth 1.0 nasce da una discussione tra Blaine Cook e Larry Halff
- 2010: RFC 5849 pubblica ufficialmente OAuth 1.0
- 2012: RFC 6749 pubblica OAuth 2.0 (Framework, non Protocollo)
- 2012: Eran Hammer si ritira dal WG OAuth 2.0
- 2015: RFC 7636 introduce PKCE (per app mobile)
- 2019: RFC 8693 Token Exchange, RFC 8628 Device Flow
- 2021: RFC 9126 PAR, OAuth 2.1 draft
- 2024: DPoP (RFC 9449), OAuth 2.1 continua a maturare

Confronto tra Authorization Server: Google, Okta, Auth0, Keycloak

Scegliere il giusto Authorization Server è una decisione architetturale importante. I fattori principali sono: deployment model (cloud vs on-premise), complessità, costi, feature set e conformità agli standard.

Google Identity Platform (accounts.google.com) è l'AS più usato al mondo per autenticazione consumer. Supporta OIDC completo, offre MFA robusta, ha SLA elevati e JWKS che cambiano regolarmente (ogni 6-24 ore circa). Limitazioni: solo per account Google, personalizzazione della schermata di consenso limitata, costi per volume alto.

Okta è il leader enterprise nell'Identity as a Service (IDaaS). Offre OIDC/OAuth 2.0 completo, SCIM per provisioning, Universal Directory, MFA avanzata (Okta Verify, WebAuthn), e integrazioni prebuilt con migliaia di applicazioni. Costo: significativo per uso enterprise. Ideale per: grandi organizzazioni con molte applicazioni e requisiti di conformità.

Auth0 (ora Cloudflare/Okta company) è popolare per gli sviluppatori. Eccellente DX (Developer Experience), SDK per molti linguaggi, Actions (regole JavaScript), Universal Login personalizzabile. Il piano gratuito (7.500 utenti attivi/mese) è generoso. Ideale per: startup, SaaS, applicazioni consumer.

Keycloak (Red Hat, ora CNCF) è il leader open source. Deploy on-premise o cloud privato, zero costi di licenza, supporto LDAP/Active Directory nativo, SSO aziendale, configurazione molto flessibile. Richiede gestione dell'infrastruttura. Ideale per: ambienti industriali, settore pubblico, chi non può usare cloud esterni.

Microsoft Azure AD (Entra ID) è obbligatorio in ecosistemi Microsoft. Integrazione nativa con Office 365, Teams, SharePoint, Azure. Supporta B2B e B2C. Per ambienti misti Windows/Linux con ADFS legacy, è spesso la scelta naturale.

- Google Identity: ottimo per login consumer, meno per enterprise
- Okta: leader enterprise IDaaS, costoso ma completo
- Auth0: eccellente DX, piano gratuito generoso per sviluppatori
- Keycloak: open source, on-premise, ideale per industria e PA
- Azure AD/Entra ID: obbligatorio per ecosistemi Microsoft

WebAuthn e FIDO2 — Il Futuro dell'Autenticazione Forte

WebAuthn (Web Authentication API, W3C 2019) e FIDO2 stanno rivoluzionando l'autenticazione forte. Invece di password o OTP, WebAuthn usa la crittografia a chiave pubblica per legare l'autenticazione al dispositivo fisico. In combinazione con OAuth 2.0/OIDC, WebAuthn abilita autenticazione passwordless di livello enterprise.

Il flusso funziona così: l'utente si registra sul proprio dispositivo (laptop, smartphone, hardware key come YubiKey). Il dispositivo genera una coppia di chiavi: la chiave privata rimane nel secure enclave del dispositivo, mai accessibile al sistema operativo o alle applicazioni. La chiave pubblica viene registrata sul server.

Al login, il server invia una 'challenge' casuale. Il dispositivo la firma con la chiave privata (richiedendo biometria o PIN dell'utente). Il server verifica con la chiave pubblica. Non c'è mai nessuna password da rubare, né phishing possibile (la sfida è legata al dominio del sito).

Come si integra con OAuth 2.0: WebAuthn è usato come meccanismo di autenticazione nell'Authorization Server (Keycloak, Okta, Auth0 lo supportano tutti). L'utente fa WebAuthn con il Keycloak, poi Keycloak emette il token OAuth/OIDC. Il client OAuth non sa nemmeno che l'utente ha

usato WebAuthn: vede solo il codice di autorizzazione e poi i token.

Passkeys (2022, Apple/Google/Microsoft) sono WebAuthn sincronizzati nel cloud. Le chiavi private sono nel portachiavi iCloud/Google Password Manager, accessibili su tutti i dispositivi dell'utente. Mantengono la sicurezza di FIDO2 con la comodità dell'accesso multi-dispositivo. Keycloak supporta Passkeys dal 2024.

- WebAuthn: autenticazione con chiave pubblica, zero password
- Secure enclave: la chiave privata non lascia mai il dispositivo
- Anti-phishing: la challenge e' legata al dominio, niente phishing
- Integrazione OAuth: WebAuthn come metodo di autenticazione nell'AS
- Passkeys: WebAuthn sincronizzato tra dispositivi (iCloud, Google)

Guida Implementativa Completa

Progettare un Sistema OAuth 2.0 da Zero

Progettare un sistema OAuth 2.0 richiede decisioni architetturali che avranno impatto sulla sicurezza, sulla scalabilità e sulla manutenibilità per anni. Questa sezione guida attraverso il processo decisionale passo per passo.

Passo 1: Identifica i tuoi attori

Prima di scrivere una riga di codice, devi identificare chiaramente chi è il Resource Owner, chi è il Client, chi è il Resource Server e chi è l'Authorization Server nel tuo sistema. In molti sistemi, la risposta non è ovvia. Per esempio, in un sistema di automazione industriale: il Resource Owner può essere sia l'operatore umano che il sistema SCADA stesso (nel caso di comunicazione M2M). Il Client può essere l'HMI, il sistema SCADA, un'app mobile per gli operatori, o un sistema di analytics. Il Resource Server può essere l'API dei dati storici, l'API dei comandi, o entrambi. L'Authorization Server è tipicamente Keycloak on-premise o Azure AD. Definire questi ruoli con precisione prima di iniziare previene errori architetturali costosi da correggere in seguito.

Passo 2: Scegli i grant type appropriati per ogni client

Non tutti i client usano lo stesso grant type. In un sistema industriale tipico coesistono: - HMI web: Authorization Code Flow + PKCE + OIDC - App mobile operatori: Authorization Code Flow + PKCE - SCADA/PLC: Client Credentials (M2M, nessun utente) - CLI di manutenzione: Device Flow (tecnici in campo senza browser) - Microservizi interni: Client Credentials La regola d'oro: se c'è un utente umano che si autentica, usa Authorization Code. Se è machine-to-machine senza utente, usa Client Credentials. Se il dispositivo non ha browser, usa Device Flow.

Passo 3: Definisci la struttura degli scope

Gli scope sono il cuore della granularità di accesso. Una buona progettazione degli scope considera: il principio del minimo privilegio (scope piccoli e specifici), la leggibilità (nomi descrittivi per la schermata di consenso), la stabilità (gli scope non dovrebbero cambiare frequentemente), e la compatibilità con standard esistenti (usa scope OIDC standard dove possibile). Schema consigliato per sistemi industriali: resource:action — es. telemetry:read, alarms:create, control:write Evita scope troppo ampi come 'admin' o 'all': preferisci 'users:read users:write' separati.

Passo 4: Pianifica la gestione del ciclo di vita dei token

Il ciclo di vita dei token deve essere pianificato: quanto durano gli access token? (raccomandazione: 5-15 minuti in sistemi critici, fino a 1 ora in sistemi consumer). I refresh token scadono? (raccomandazione: sì, con rotation obbligatoria). Quando si revocano i token? (al logout, al cambio password, al sospetto di compromissione). Come si implementa il logout globale? (back-channel logout per SSO). Come si gestisce la revoca di emergenza? (endpoint admin per revocare tutti i token di un utente). Tutte queste decisioni devono essere prese prima dell'implementazione.

Passo 5: Sicurezza dei segreti (client_secret)

Il `client_secret` è la chiave privata dell'applicazione. La sua protezione è critica: in produzione i segreti devono essere in un vault (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager), mai in file `.env` committati in Git, mai hardcoded nel codice, mai in variabili d'ambiente non cifrate nei sistemi di orchestrazione. Il `client_secret` deve essere ruotato periodicamente (raccomandazione: ogni 90 giorni) e immediatamente se sospetto di compromissione. Per i client pubblici (app mobile, SPA) il `client_secret` non esiste: PKCE è il sostituto crittografico.

Monitoraggio e Observability OAuth

Il monitoraggio del sistema OAuth è essenziale sia per la sicurezza che per il debugging. Un sistema OAuth ben monitorato permette di rilevare attacchi, identificare errori di configurazione e misurare le performance.

Metriche chiave da monitorare

Le metriche più importanti per un sistema OAuth sono: Token emission rate (token/secondo emessi dal token endpoint): un picco improvviso può indicare un attacco brute-force o una misconfigurazione. Token validation rate (validazioni/secondo nei Resource Server): permette di dimensionare l'infrastruttura e rilevare anomalie. Token rejection rate (percentuale di token rifiutati): un aumento improvviso può indicare token scaduti per errore di configurazione o tentativo di uso di token rubati. Refresh token usage (frequenza dei refresh): se molto più alta del normale, potrebbe indicare access token con vita troppo breve. Error rate per tipo di errore (`invalid_client`, `invalid_grant`, ecc.): ogni tipo di errore ha implicazioni diverse sulla sicurezza.

Logging di sicurezza

Ogni evento OAuth deve essere loggato con contesto sufficiente per il forensic in caso di incidente: chi ha richiesto il token (`client_id`, `user_id`), quando (timestamp preciso, timezone), da dove (IP sorgente, user agent), con quali permessi (scope, `grant_type`), con quale esito (successo, errore e tipo), identificatore univoco del token (jti per correlazione). I log devono essere immutabili (append-only), centralizzati in un SIEM, conservati per almeno 12 mesi (GDPR, NIS2), e protetti da accesso non autorizzato.

Alert e anomaly detection

Alert da configurare sul sistema OAuth: - Tasso di errori `invalid_client` superiore alla soglia: possibile attacco - Login da paese inusuale: anomalia geografica - Numero di token attivi per utente superiore al normale: sessioni duplicate - Refresh token invalidation improvvisa: possibile token theft (rotation avvisata) - Latenza token endpoint > soglia: degradazione performance o attacco - Certificati TLS in scadenza entro 30 giorni: prevenire outage

Scenari di Errore e Recovery

I sistemi OAuth devono essere progettati per gestire gracefully i fallimenti dei componenti. L'Authorization Server è infrastruttura critica: la sua indisponibilità deve avere impatto minimo sulle applicazioni.

AS non disponibile — strategie di mitigazione

Se l'Authorization Server non è disponibile: I JWT già emessi rimangono validi fino alla loro scadenza naturale. Questo è un vantaggio dei JWT self-contained rispetto ai token opachi: non richiedono round-trip all'AS per la validazione. Strategia raccomandata: access token con vita breve (5-15 min) per i sistemi critici, con la consapevolezza che durante un outage dell'AS le sessioni attive continuano a funzionare fino alla scadenza dei token correnti. Per sistemi critici (impianti industriali): implementa un 'emergency access mode' con token di emergenza pre-generati (offline tokens), usati solo se l'AS non è raggiungibile per più di N minuti.

Token rubato — procedura di risposta

Se si sospetta che un token sia stato compromesso:

1. Revoca immediata tramite revocation endpoint (RFC 7009)
2. Revoca di tutti i token del client interessato nell'AS
3. Disabilita temporaneamente il client nell'AS se necessario
4. Analizza i log per determinare l'entità del breach
5. Notifica gli utenti interessati (GDPR: 72 ore se breach di dati personali)
6. Ruota immediatamente il client_secret
7. Aggiorna le chiavi JWKS (cambia la coppia di chiavi RSA usata per firmare)
8. Implementa monitoring rafforzato per i giorni successivi

La rotation automatica del refresh token (già implementata) limita la finestra di abuso: un refresh token usato genera un nuovo RT e invalida il vecchio. Se un attaccante usa il RT compromesso, l'utente legittimo riceverà un errore al prossimo refresh, segnalando il breach.

Ecosistema Python per OAuth 2.0

L'ecosistema Python per OAuth 2.0 è ricco e maturo. Questa sezione fornisce una panoramica dettagliata delle librerie, dei framework e degli strumenti disponibili.

Librerie Client OAuth

Libreria	pip install	Uso principale	Note
requests-oauthlib	requests-oauthlib	Client OAuth generico con requests	request è usata per script
Authlib	authlib	Full stack: client + server	Flask/Django/FastAPI
google-auth	google-auth google-auth-oauthlib	Google APIs	Ufficiale Google
msal	msal	Microsoft Azure AD/Entra ID	Ufficiale Microsoft
python-keycloak	python-keycloak	Keycloak admin + OIDC	Admin API + user auth
social-auth-core	social-auth-core	Social login multi-provider	Django/Flask social
fastapi-users	fastapi-users[oauth]	Auth completa per FastAPI	Opinionated, rapido
django-allauth	django-allauth	Social auth per Django	40+ provider
flask-dance	flask-dance	Social auth per Flask	Semplice setup
oauthlib	oauthlib	RFC implementation bassa	Base per requests-oauthlib

Librerie JWT

Libreria	pip install	Pro	Contro
PyJWT	PyJWT cryptography	Semplice, popolare	Meno funzionalità JWE
python-jose	python-jose[cryptography]	JWT + JWK + JWE + JWS	Più verboso
Authlib jose	authlib	Parte di authlib, completo	Dipendenza grande
jwcrypto	jwcrypto	JWK, JWE, JWS completo	Meno documentato

Strumenti di Debug OAuth

Il debugging di un'implementazione OAuth può essere difficile. Questi strumenti aiutano a ispezionare richieste, token e flussi.

■ Strumenti di debug OAuth in Python

```
# Strumenti di debug per sviluppo OAuth
```

```
# 1. jwt.io – decodifica JWT online (non usare con token reali di produzione!)
# https://jwt.io
```

```
# 2. Decodifica JWT in Python senza validare (solo per debug!)
import jwt, json, base64
```

```
def decode_jwt_unsafe(token):
    """DEBUG ONLY: decodifica JWT senza verifica firma."""
```

```
parts = token.split('.')
for i, part in enumerate(['Header', 'Payload']):
    padded = parts[i] + '=' * (4 - len(parts[i]) % 4)
    decoded = json.loads(base64.urlsafe_b64decode(padded))
    print(f"\n=== {part} ===")
    print(json.dumps(decoded, indent=2))

# 3. Intercepta richieste HTTP OAuth con proxy locale (mitmproxy)
# pip install mitmproxy
# mitmproxy --mode regular --listen-port 8888
# Poi configura: export HTTPS_PROXY=http://localhost:8888

# 4. Logging dettagliato di requests
import logging, requests
logging.basicConfig(level=logging.DEBUG)

from http.client import HTTPConnection
HTTPConnection.debuglevel = 1 # Logga tutto il traffico HTTP

# 5. Verifica claims JWT dal terminale
token = "eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1c2VyXzEyMyJ9.SIGNATURE"
decode_jwt_unsafe(token)

# 6. Verifica JWKS endpoint
import requests
jwks = requests.get('https://accounts.google.com/.well-known/jwks.json').json()
print(f"Numero di chiavi pubbliche Google: {len(jwks['keys'])}")
for k in jwks['keys']:
    print(f"  kid: {k['kid']}, alg: {k.get('alg', 'N/A')}, use: {k.get('use', 'N/A')}")
```

Deployment OAuth in Produzione

Il deployment di un sistema OAuth in produzione richiede attenzione a molti dettagli oltre al codice: infrastruttura, alta disponibilità, monitoraggio e manutenzione.

Alta Disponibilità dell'Authorization Server

L'Authorization Server è un singolo punto di fallimento critico: se non è disponibile, nessun utente può autenticarsi. Per sistemi ad alta disponibilità (HA), occorre eseguire almeno 2-3 istanze dell'AS dietro un load balancer. Keycloak supporta il clustering con Infinispan (cache distribuita) per condividere sessioni tra nodi. Le sessioni OAuth sono statefull: un utente che si autentica su nodo A deve poter fare il refresh su nodo B. Questo richiede sessioni condivise in un database (PostgreSQL per Keycloak) o cache distribuita (Infinispan/Redis).

■ Keycloak HA con Docker Compose

```
# Keycloak HA con Docker Compose (esempio semplificato)
# In produzione usa Kubernetes con l'operatore Keycloak

# docker-compose.yml
version: '3.8'
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data

  keycloak_1:
    image: quay.io/keycloak/keycloak:latest
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://postgres:5432/keycloak
      KC_DB_USERNAME: keycloak
      KC_DB_PASSWORD: ${DB_PASSWORD}
      KC_HOSTNAME: auth.factory.com
      KC_CACHE: ispn # Infinispan clustering
      KC_CACHE_STACK: kubernetes # 0 'udp' per Docker
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: ${ADMIN_PASSWORD}
    depends_on: [postgres]
    command: start

  keycloak_2:
    # Identico a keycloak_1 – clustering automatico
    <<: *keycloak_base

  nginx:
    image: nginx:alpine
    volumes: [./nginx.conf:/etc/nginx/nginx.conf]
    ports: ['443:443']
    # nginx.conf: upstream con keycloak_1 e keycloak_2, sticky sessions
    # sticky sessions non necessarie con clustering Infinispan correttamente configurato
```

```
volumes:
  pgdata:
```

Configurazione NGINX come Reverse Proxy OAuth

■ NGINX reverse proxy per Keycloak con sicurezza

```
# nginx.conf - reverse proxy con header sicurezza OAuth
upstream keycloak_cluster {
    least_conn;    # Bilanciamento round-robin con minore carico
    server keycloak_1:8443;
    server keycloak_2:8443;
    keepalive 32;  # Connessioni persistenti al backend
}

upstream api_servers {
    server api_1:8000;
    server api_2:8000;
    server api_3:8000;
}

server {
    listen 443 ssl http2;
    server_name auth.factory.com;

    ssl_certificate      /etc/ssl/factory.crt;
    ssl_certificate_key  /etc/ssl/factory.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers          ECDHE-RSA-AES256-GCM-SHA384:ECDHE-RSA-AES128-GCM-SHA256;
    ssl_prefer_server_ciphers on;
    ssl_session_cache    shared:SSL:10m;

    # Header di sicurezza
    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Frame-Options DENY always;
    add_header X-Content-Type-Options nosniff always;
    add_header Content-Security-Policy "default-src 'self'" always;

    # Rate limiting per il token endpoint
    limit_req_zone $binary_remote_addr zone=token:10m rate=10r/s;

    location /realms/ {
        limit_req zone=token burst=20 nodelay;
        proxy_pass https://keycloak_cluster;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Kubernetes Deployment con Keycloak Operator

■ Keycloak su Kubernetes con HA e autoscaling

```
# Keycloak su Kubernetes con l'operatore ufficiale
# kubectl apply -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/
# 23.0.0/kubernetes/keycloaks.k8s.keycloak.org-v1.yml

apiVersion: k8s.keycloak.org/v2alpha1
```

```
kind: Keycloak
metadata:
  name: keycloak-factory
  namespace: auth
spec:
  instances: 3 # 3 repliche per HA
  db:
    vendor: postgres
    host: postgres-service
    database: keycloak
    usernameSecret:
      name: keycloak-db-secret
      key: username
    passwordSecret:
      name: keycloak-db-secret
      key: password
  http:
    tlsSecret: factory-tls-secret # TLS termination in Keycloak
  hostname:
    hostname: auth.factory.com
  resources:
    requests:
      memory: "512Mi"
      cpu: "500m"
    limits:
      memory: "1Gi"
      cpu: "1000m"
  additionalOptions:
    - name: cache
      value: ispn # Infinispan clustering tra pod
    - name: cache-stack
      value: kubernetes # Scoperta automatica pod K8s
---
# HorizontalPodAutoscaler per scalabilità automatica
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: keycloak-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: StatefulSet
    name: keycloak-factory
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```


Analisi Tecnica degli RFC OAuth 2.0

Questa sezione analizza in dettaglio i principali RFC che compongono l'ecosistema OAuth 2.0, con riferimento ai punti più importanti e spesso fraintesi.

RFC 6749 — Analisi Punto per Punto

RFC 6749 è il documento fondativo di OAuth 2.0. Pubblicato nel 2012, definisce il framework generale. Alcuni punti spesso ignorati o malinterpretati:

Section 2.1 — Tipi di Client

RFC 6749 distingue tra 'confidential client' (in grado di mantenere segreti, es. web server) e 'public client' (non in grado, es. app mobile, SPA). Questa distinzione determina se il `client_secret` è obbligatorio. Un errore comune è trattare le SPA come confidential client e hardcodare il `client_secret` nel JavaScript: il `client_secret` è visibile a chiunque ispeziona il sorgente. Le SPA sono public client e devono usare PKCE.

Section 3.1.2 — Redirect URI

RFC 6749 section 3.1.2 specifica che il `redirect_uri` deve essere pre-registrato e confrontato esattamente. Tuttavia la sezione 4.1.1 dice che si può omettere il `redirect_uri` nella richiesta se è stato pre-registrato uno solo. Questa flessibilità ha causato vulnerabilità: alcuni AS facevano prefix-match invece di exact-match. OAuth 2.1 richiede exact-match obbligatorio.

Section 4.4 — Client Credentials

RFC 6749 section 4.4 specifica il Client Credentials Grant. Un dettaglio importante: il refresh token è facoltativo per questo grant type. La maggior parte degli AS non emette refresh token per Client Credentials perché il client può semplicemente richiedere un nuovo access token con le sue credenziali.

Section 5.1 — Token Response

RFC 6749 section 5.1 richiede che la risposta includa Cache-Control: no-store e Pragma: no-cache. Questi header sono OBBLIGATORI per prevenire che i token vengano memorizzati in caches HTTP intermedie. Molte implementazioni li omettono per errore.

Section 10.12 — CSRF

RFC 6749 section 10.12 descrive l'attacco CSRF e richiede il parametro 'state'. Il testo usa 'SHOULD' (raccomandato) non 'MUST' (obbligatorio), causando implementazioni senza state. OAuth 2.1 ha corretto in 'MUST'.

RFC 6750 — Bearer Token Usage

RFC 6750 definisce come usare i Bearer token nelle richieste HTTP. Definisce tre metodi di invio del token, con preferenza esplicita per il primo:

Metodo	Formato	Raccomandato?
Authorization header	Authorization: Bearer <token>	Sì — metodo preferito

Form body	access_token=<token> (POST body)	Solo se no header
Query parameter	?access_token=<token> (nell'URL)	DEPRECATED in OAuth 2.1

■ Il query parameter è il metodo più pericoloso: il token appare nei log del server web, nella cronologia del browser, negli header Referer inviati a siti terzi. OAuth 2.1 vieta questo metodo.

RFC 7636 — PKCE in Dettaglio

PKCE (RFC 7636, 2015) fu introdotto per risolvere l'Authorization Code Interception Attack nelle app mobile. La specifica è elegante nella sua semplicità:

Parametro	Dove	Valore	Note
code_verifier	Client (segreto)	Stringa casuale 43-128 char	MAI inviato nell'URL
code_challenge	URL di autorizzazione	BASE64URL(SHA256(verifier))	Hash, non il segreto
code_challenge_method	URL di autorizzazione	'S256'	Non usare 'plain'
code_verifier	Token endpoint	Il verifier originale	Verificato dall'AS

■ Esiste un code_challenge_method 'plain' che non fa hashing. Non usarlo mai: se l'URL è intercettato, il verifier è esposto. Usa sempre 'S256'.

RFC 8693 — Token Exchange

Token Exchange (RFC 8693, 2019) permette di scambiare un token per un altro con caratteristiche diverse. Questo è utile in scenari di impersonation e delegazione in sistemi microservizi.

■ Token Exchange RFC 8693

```
# Token Exchange – caso d'uso: microservizio A chiama microservizio B per conto dell'utente
# A riceve il token dell'utente, lo scambia per un token con audience=serviceB
```

```
import requests
```

```
def exchange_token_for_service(token: str, target_service: str,
                               token_endpoint: str, client_id: str,
                               client_secret: str) -> str:
    """
    RFC 8693 Token Exchange.
    Scambia il token dell'utente per uno specifico per il servizio target.
    """
    resp = requests.post(token_endpoint, data={
        'grant_type': 'urn:ietf:params:oauth:grant-type:token-exchange',
        'subject_token': token,
        'subject_token_type': 'urn:ietf:params:oauth:token-type:access_token',
        'requested_token_type': 'urn:ietf:params:oauth:token-type:access_token',
        'audience': target_service, # Audience del servizio target
        'scope': 'read',
    }, auth=(client_id, client_secret))

    if resp.status_code == 200:
        return resp.json()['access_token']
```

```
        raise Exception(f"Token exchange failed: {resp.text}")

# ServiceA ottiene il token utente, lo scambia per un token per ServiceB
user_token = request.headers.get('Authorization', '').replace('Bearer ', '')
service_b_token = exchange_token_for_service(
    user_token, 'service-b.factory.internal',
    'https://as.factory.com/token', 'service-a', 'secret_a')

# Ora chiama ServiceB con il token specifico
resp = requests.get('https://service-b.factory.internal/data',
                    headers={'Authorization': f'Bearer {service_b_token}'})
```

Scenari di Attacco Reali e Mitigazioni

Questa sezione analizza scenari di attacco reali documentati nel corso degli anni, con dettagli tecnici su come funzionano e come difendersi.

Attacco 1: Open Redirect via redirect_uri

Un attaccante registra l'app legittima su un AS che fa prefix-match del redirect_uri. Se redirect_uri=https://myapp.com/ è registrato, e l'AS accetta https://myapp.com.attacker.com/ come match, l'attaccante può costruire un URL: https://as.example.com/authorize?client_id=myapp&redirect_uri=https://myapp.com.attacker.com/ L'utente autentica, l'AS reindirizza il code al sito dell'attaccante. L'attaccante scambia il code per il token impersonando l'utente. Impatto: account takeover completo. Mitigazione: exact-match del redirect_uri (obbligatorio in OAuth 2.1). Alcuni AS (Google) fanno exact-match dal 2015. Verifica la policy del tuo AS.

- Usa AS con exact-match redirect_uri
- Registra URI specifici, non pattern o prefissi
- Aggiorna all'ultima versione dell'AS

Attacco 2: Mix-Up Attack

Scenario: un attaccante controlla un AS malevolo (AS-B) che usa lo stesso client_id del client registrato sull'AS legittimo (AS-A). Il client riceve un code da AS-B ma, confuso, lo invia ad AS-A. AS-A potrebbe accettarlo se non distingue quale AS ha emesso il code. Questo attacco è particolarmente pericoloso in scenari multi-AS (federazione). Mitigazione: il client deve verificare che l'iss nel token corrisponda all'AS previsto. Con OIDC il server metadata (issuer) previene questo. PAR (Pushed Authorization Requests) previene anche questo: il code è legato alla specifica richiesta PAR, non ambiguo tra AS diversi.

- Verifica sempre iss nel token
- Usa OIDC discovery per identificare l'AS
- Considera PAR in scenari multi-AS

Attacco 3: Subdomain Takeover + OAuth

Se redirect_uri=https://staging.myapp.com/callback e il sottodominio staging.myapp.com scade o viene eliminato, un attaccante può registrare staging.myapp.com su un hosting cloud e ricevere tutti i redirect OAuth. Questo attacco ha colpito applicazioni reali (GitHub, altri provider). Impatto: chiunque conosca il client_id e questo URI compromesso può ricevere authorization code reali degli utenti. Mitigazione: monitoraggio DNS dei sottodomini, rimozione immediata degli URI non più in uso, uso di domini stabili per i callback OAuth.

- Monitora tutti i DNS dei sottodomini usati come redirect_uri
- Rimuovi immediatamente gli URI obsoleti dalla registrazione
- Usa staging separato con client diverso

Attacco 4: Authorization Code Phishing

Un attaccante crea un'app fasullo che richiede scope ampi e tenta di far autenticare l'utente su un AS legittimo (Google, Microsoft). L'utente vede una schermata di consenso legittima, non il sito dell'attaccante. Se concede il consenso, il code viene inviato all'attaccante. Questo è usato negli attacchi 'consent phishing' documentati dal 2017 in poi. Microsoft ha bloccato migliaia di app malevole registrate su Azure AD. Mitigazione: education degli utenti, policy 'admin consent' (solo l'admin dell'organizzazione può approvare nuove app), revisione periodica delle app che hanno accesso ai propri account.

- Abilita 'admin consent required' per app enterprise
- Educare gli utenti a verificare il nome dell'app nella schermata di consenso
- Revisione periodica dell'accesso delle app di terze parti

Scenari Avanzati e Specializzati

OAuth 2.0 per Sistemi Multi-Tenant

I sistemi multi-tenant dove ogni cliente (tenant) ha il proprio spazio dati presentano sfide specifiche per OAuth 2.0. I principali pattern architetturali sono:

Pattern	AS	Vantaggio	Svantaggio
Un AS globale, tenant via scope	Uno	Semplice	Scope complessi
Un realm per tenant (Keycloak)	Uno (multi-realm)	Isolamento completo	Overhead gestione
AS separato per tenant	N (uno/tenant)	Massimo isolamento	Costi infrastruttura
Federation (Azure AD multi-tenant)	Federati	SSO aziendale tenant	Complessità federation

Multi-tenant OAuth con Keycloak realm separati

```
# Gestione multi-tenant con Keycloak – realm per tenant
# Ogni cliente industriale ha il proprio realm Keycloak isolato

from keycloak import KeycloakAdmin, KeycloakOpenID

class MultiTenantAuth:
    """Gestione autenticazione per sistema SaaS industriale multi-tenant."""
    def __init__(self, keycloak_base: str, admin_user: str, admin_pass: str):
        self.kc_base = keycloak_base
        self._admin_user = admin_user
        self._admin_pass = admin_pass

    def provision_tenant(self, tenant_id: str, tenant_name: str) -> dict:
        """Crea un nuovo realm Keycloak per il tenant (nuovo cliente)."""
        admin = KeycloakAdmin(server_url=self.kc_base,
                               username=self._admin_user,
                               password=self._admin_pass,
                               realm_name='master')

        # Crea realm isolato per il tenant
        admin.create_realm({
            'realm': tenant_id,
            'displayName': tenant_name,
            'enabled': True,
            'sslRequired': 'external',
            'accessTokenLifespan': 600, # 10 min per sicurezza industriale
            'refreshTokenMaxReuse': 0,
            'revokeRefreshToken': True,
        })

        # Crea client SCADA per questo tenant
        admin.realm_name = tenant_id
        secret = generate_secure_secret()
        admin.create_client({
            'clientId': f'scada_{tenant_id}',
            'enabled': True, 'secret': secret,
            'standardFlowEnabled': True,
            'serviceAccountsEnabled': True,
        })

        return {'realm': tenant_id, 'client_id': f'scada_{tenant_id}'}
```

```

        'client_secret': secret}

    def get_oidc_for_tenant(self, tenant_id: str) -> KeycloakOpenID:
        return KeycloakOpenID(
            server_url=self.kc_base, realm_name=tenant_id,
            client_id=f'scada_{tenant_id}',
            client_secret_key=get_secret_from_vault(tenant_id))

import secrets
def generate_secure_secret(): return secrets.token_urlsafe(48)
def get_secret_from_vault(tid): return 'vault_fetched_secret' # stub

```

OAuth 2.0 con gRPC e Protocol Buffers

Nei sistemi moderni i microservizi comunicano spesso tramite gRPC oltre che REST. OAuth 2.0 si integra con gRPC tramite i metadata (equivalenti degli header HTTP).

■ gRPC OAuth interceptor

```

# gRPC interceptor per validazione JWT
import grpc
import jwt

class OAuthInterceptor(grpc.ServerInterceptor):
    """Valida il Bearer token JWT nei metadata gRPC per ogni chiamata."""
    def __init__(self, jwks_cache: JWKSCache, issuer: str, audience: str):
        self.jwks = jwks_cache
        self.issuer = issuer
        self.audience = audience

    def intercept_service(self, continuation, handler_call_details):
        # Estrai token dai metadata gRPC
        metadata = dict(handler_call_details.invocation_metadata or [])
        auth = metadata.get('authorization', '')
        if not auth.lower().startswith('bearer '):
            return self._abort(grpc.StatusCode.UNAUTHENTICATED, 'Bearer token required')
        token = auth[7:]
        try:
            hdr = jwt.get_unverified_header(token)
            key = self.jwks.get(hdr.get('kid'))
            jwt.decode(token, key, algorithms=['RS256'],
                       audience=self.audience, issuer=self.issuer)
        except jwt.ExpiredSignatureError:
            return self._abort(grpc.StatusCode.UNAUTHENTICATED, 'Token expired')
        except jwt.InvalidTokenError as e:
            return self._abort(grpc.StatusCode.UNAUTHENTICATED, str(e))
        return continuation(handler_call_details)

    def _abort(self, code, details):
        def h(req, ctx): ctx.abort(code, details)
        return grpc.unary_unary_rpc_method_handler(h)

# Client gRPC con Bearer token
import grpc
def make_grpc_call(channel, access_token):
    stub = TelemetryServiceStub(channel)
    metadata = [('authorization', f'Bearer {access_token}')]
    return stub.SendTelemetry(request, metadata=metadata, timeout=5)

```

OAuth 2.0 per MQTT e Protocolli IoT

MQTT è il protocollo di messaging standard per IoT. Sebbene MQTT non supporti nativamente OAuth 2.0, i broker MQTT moderni (EMQX, HiveMQ, Mosquitto con plugin) supportano autenticazione tramite JWT.

■ MQTT con autenticazione JWT OAuth

```
# MQTT con autenticazione JWT (Bearer token come password)
# Il broker EMQX o HiveMQ verificano il JWT prima di permettere publish/subscribe
```

```
import paho.mqtt.client as mqtt
import ssl, time
```

```
class MqttOAuthClient:
    """Client MQTT che usa JWT OAuth come password di autenticazione."""
    def __init__(self, broker, port, token_manager: TokenManager):
        self.broker = broker
        self.port = port
        self.tokenmgr = token_manager
        self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
        self.client.tls_set(tls_version=ssl.PROTOCOL_TLSv1_2)
        self.client.on_connect = self._on_connect
        self.client.on_disconnect = self._on_disconnect
        self.client.on_message = self._on_message

    def connect(self):
        token = self.tokenmgr.get_valid_token()
        # MQTT: username = client_id (o vuoto), password = Bearer token JWT
        self.client.username_pw_set(
            username='mqtt_client',
            password=token # Il broker valida il JWT come password
        )
        self.client.connect(self.broker, self.port, keepalive=60)

    def publish_telemetry(self, station: str, data: dict):
        """Pubblica telemetria con token rinnovato automaticamente."""
        if self.tokenmgr._expiry < time.time() + 30:
            # Token quasi scaduto: riconnetti con nuovo token
            self.client.disconnect()
            time.sleep(0.5)
            self.connect()
        import json
        topic = f"factory/{station}/telemetry"
        payload = json.dumps(data)
        result = self.client.publish(topic, payload, qos=1)
        return result.rc == mqtt.MQTT_ERR_SUCCESS

    def _on_connect(self, client, userdata, flags, rc, props=None):
        if rc == 0: print("MQTT connesso")
        elif rc == 5: print("MQTT: autenticazione fallita – token non valido")

    def _on_disconnect(self, client, userdata, rc, props=None):
        if rc != 0:
            print(f"MQTT disconnessione inattesa (rc={rc}), riconnessione...")

    def _on_message(self, client, userdata, msg):
        print(f"Ricevuto su {msg.topic}: {msg.payload.decode()}")
```



```
# Utilizzo per impianto acque
auth = ClientCredentialsAuth('https://as.factory.com/token',
                             'plc_mqtt_01', 'secret', 'mqtt:publish')
tokenmgr = TokenManager('https://as.factory.com/token', 'plc_mqtt_01', 'secret')
tokenmgr.set_initial_tokens(auth.get_token(), None, 3600)
mqtt_client = MqttOAuthClient('mqtt.factory.com', 8883, tokenmgr)
mqtt_client.connect()
mqtt_client.publish_telemetry('ST-01', {'ph': 7.2, 'flow': 120.5})
```

Strumenti e Risorse per OAuth 2.0

Strumenti di Test e Debug

Strumento	Tipo	Uso
jwt.io	Web	Decodifica e verifica JWT online (non produzione)
Postman	Desktop	Test endpoint OAuth con OAuth 2.0 helper integrato
Insomnia	Desktop	REST client con supporto OAuth 2.0
mitmproxy	CLI/Web	Proxy per intercettare e analizzare traffico OAuth
OAuth Debugger	Web	Simula flussi OAuth con provider reali (oauthdebugger.com)
Keycloak Dev Console	Web	AS open source con UI per test e configurazione
Auth0 Playground	Web	Test flussi OIDC/OAuth con Auth0
OpenAPI/Swagger	Standard	Documenta endpoint API con schema OAuth (securitySchemes)
JWT CLI (tool)	CLI	Genera e verifica JWT da terminale (npm: jsonwebtoken)
jq	CLI	Analizza risposte JSON OAuth dal terminale

Risorse di Apprendimento e Riferimento

Risorsa	Tipo	Contenuto
oauth.net	Web ufficiale	Tutti gli RFC e news OAuth
openid.net	Web ufficiale	Specifiche OIDC e conformance
RFC Editor (rfc-editor.org)	RFC	Testo completo di tutti gli RFC
The OAuth 2.0 Simplified (aaronparecki.com)	Libro online	Guida pratica di Aaron Parecki
Auth0 Blog	Blog	Articoli tecnici OAuth/OIDC pratici
Okta Developer Blog	Blog	Tutorial OAuth con Java, Python, JS
OAuth 2.0 Security BCP (RFC 9700)	RFC	Best Current Practice per sicurezza
OWASP API Security Top 10	Standard	Include vulnerabilità OAuth comuni
Python documentation — ssl module	Doc	TLS in Python — base di OAuth

Checklist Finale Prima del Go-Live

- ☐ HTTPS su tutti gli endpoint (AS, API, callback)
- ☐ client_secret in vault (non in .env committati o hardcoded)
- ☐ PKCE implementato per tutti i client pubblici
- ☐ state parameter presente e verificato
- ☐ redirect_uri exact-match verificato nell'AS

- [] Access token con vita ≤ 1 ora (5-15 min per sistemi critici)
- [] Refresh token rotation abilitata nell'AS
- [] Revoca token al logout (revocation endpoint chiamato)
- [] Validazione JWT: iss, aud, exp, alg specificato (non dall'header)
- [] JWKS cache implementata (non fetch per ogni validazione)
- [] Scope minimali (principio del minimo privilegio)
- [] Audit log abilitato per tutti gli eventi OAuth
- [] Monitoring e alert su error rate e latenza token endpoint
- [] Test con token scaduto, token revocato, token di altro utente
- [] Test CSRF: cosa succede se state è errato?
- [] Nessun token nei log, negli URL, nei messaggi di errore
- [] Rate limiting sul token endpoint
- [] Certificati TLS monitorati per scadenza
- [] Backup e HA per l'Authorization Server
- [] Documentazione aggiornata e procedure di incident response

Guida Enterprise e Governance OAuth 2.0

Governance e Processi per OAuth 2.0 Enterprise

In un'organizzazione enterprise, OAuth 2.0 non è solo una questione tecnica: richiede governance, processi e responsabilità chiare. Senza una struttura di governance, si rischia proliferazione incontrollata di client registrati, scope ridondanti, e segreti non ruotati.

Il primo elemento di governance è un registro centralizzato di tutte le applicazioni registrate presso l'AS. Ogni applicazione deve avere un owner identificato, una descrizione del caso d'uso, gli scope richiesti giustificati, e una data di revisione pianificata. Le applicazioni senza owner o non revisionate da più di un anno devono essere rimosse o sospese.

Il processo di on-boarding di una nuova applicazione OAuth deve includere: analisi dei requisiti di accesso (quali risorse, con quale granularità), revisione della sicurezza (grant type appropriato, PKCE, redirect URI), approvazione del responsabile sicurezza, registrazione nell'AS e nel registro, comunicazione delle credenziali al team applicativo in modo sicuro.

La revisione periodica (trimestrale o semestrale) dell'uso dei token permette di identificare applicazioni non più usate ma ancora registrate, scope richiesti ma mai effettivamente utilizzati, pattern anomali di utilizzo che potrebbero indicare compromissione.

La separazione degli ambienti è fondamentale: ogni ambiente (sviluppo, test, staging, produzione) deve avere il suo AS separato con credenziali diverse. Non usare mai le credenziali di produzione in sviluppo. I token di produzione non devono mai essere visibili in ambienti non produttivi.

Framework di Compliance e OAuth 2.0

OAuth 2.0 è rilevante per molti framework di compliance. Capire le sovrapposizioni permette di implementare OAuth 2.0 in modo che soddisfi più requisiti simultaneamente.

Framework	Area rilevante OAuth	Controllo principale
SOC 2	Access control, audit trail	Evidenza di revoca, MFA, audit log
ISO 27001	A.9 Access Control	Policy accesso, revisione privilegi
PCI DSS	Req. 7 (minimo privilegio), Req. 8 (auth)	Scope minimali, MFA per admin
HIPAA	Access control, audit controls	PHI access log, revoca immediata
GDPR	Data minimization, consent	Scope minimi, revoca consenso
NIST 800-53	IA (Identification & Auth), AC (Access Control)	MFA, token lifecycle management
CIS Controls	Control 5 (Accounts), Control 6 (Access)	Account OAuth inventario, MFA

Per SOC 2 Type II, le organizzazioni devono dimostrare che i controlli di accesso funzionano nel tempo. Con OAuth 2.0 questo significa: log completo delle autenticazioni con timestamp e outcome, evidenza di revoca dei token al termine dell'accesso o alla scadenza del contratto, MFA abilitata per tutti gli accessi alle risorse critiche, e processi documentati per la gestione delle credenziali OAuth.

ISO 27001 Annex A.9 copre la gestione degli accessi. OAuth 2.0 implementa direttamente molti dei controlli richiesti: A.9.1.2 (accesso ai servizi di rete) tramite autenticazione Bearer token, A.9.2.3

(gestione dei diritti di accesso privilegiati) tramite scope e ruoli, A.9.2.5 (revisione dei diritti di accesso) tramite log di utilizzo dei token, A.9.4.2 (procedure di log-on sicure) tramite OIDC con MFA.

PCI DSS Requirement 8 richiede autenticazione forte per tutti gli utenti. OAuth 2.0 con OIDC e MFA soddisfa questo requisito. Il Requirement 8.2 richiede gestione sicura delle credenziali: i `client_secret` devono essere trattati come password — mai condivisi, mai inviati via email, ruotati periodicamente. Il Requirement 8.8 richiede revisione delle policy di autenticazione: il registro delle app OAuth e le revisioni periodiche soddisfano questo.

OAuth Incident Response Playbook

Un playbook di incident response specifico per OAuth 2.0 permette di rispondere rapidamente e in modo coordinato in caso di compromissione di token o credenziali.

Scenario: Sospetto token leak

- 1. Identifica quale token è stato potenzialmente compromesso (access/refresh?)
- 2. Revoca immediatamente via revocation endpoint (RFC 7009)
- 3. Se refresh token: revoca tutti i token della sessione correlata
- 4. Analizza i log del resource server per il jti del token compromesso
- 5. Determina quali risorse sono state accedute nel periodo sospetto
- 6. Notifica l'utente interessato se l'accesso era per suo conto
- 7. Se `client_secret` compromesso: ruota il segreto nell'AS e nel vault
- 8. Aggiorna il Keycloak/AS per invalidare tutti i token emessi con il vecchio segreto

Scenario: Authorization Server compromesso

- 1. Isola immediatamente l'AS dalla rete esterna
- 2. Tutti i token emessi devono essere considerati non fidati
- 3. Rotazione di tutte le chiavi di firma JWT (nuovo JWKS)
- 4. Tutti i Resource Server devono ricaricare il JWKS
- 5. Tutti gli utenti devono ri-autenticarsi
- 6. Analisi forense: quali token sono stati emessi durante il periodo di compromissione?
- 7. Comunicazione agli utenti e agli stakeholder (GDPR: 72h se breach dati personali)
- 8. Patching, hardening e re-deployment dell'AS
- 9. Post-mortem entro 5 giorni

Scenario: Scoperta di implementazione insicura

- 1. Valuta l'impatto: exploit attivo o solo vulnerabilità teorica?
- 2. Se exploit attivo: segui playbook token leak
- 3. Pianifica il fix in modo che non rompa le sessioni attive
- 4. Per PKCE mancante: aggiungi PKCE preservando retrocompatibilità
- 5. Per state mancante: aggiungi state con migration graduale
- 6. Per `redirect_uri` non esatto: aggiorna la policy dell'AS
- 7. Test di regressione completo prima del deploy
- 8. Aggiorna documentazione e checklist

Training e Cultura della Sicurezza OAuth

La sicurezza di un sistema OAuth dipende tanto dalla cultura organizzativa quanto dalla tecnologia. Gli sviluppatori che non capiscono i principi fondamentali di OAuth 2.0 fanno errori che la tecnologia non può prevenire: hard-codano `client_secret` nel codice, non verificano lo `state parameter`, mettono token nell'URL.

Un programma di training efficace su OAuth 2.0 per sviluppatori deve coprire: concetti fondamentali (delegazione, non autenticazione), errori comuni e come evitarli (con esempi di codice sicuro e insicuro), policy aziendali per la gestione delle credenziali OAuth, e come segnalare una vulnerabilità sospetta.

Il codice review è uno strumento essenziale. Includere OAuth 2.0 nella checklist di security code review: il `client_secret` è mai nel codice o nei commit? Lo `state parameter` è verificato? Il `redirect_uri` è hardcoded o configurabile esternamente? I token sono memorizzati in modo sicuro? La validazione JWT specifica gli algoritmi attesi?

Threat modeling periodico del sistema OAuth permette di identificare nuovi rischi man mano che il sistema evolve. Ogni nuova integrazione OAuth deve passare attraverso un threat model rapido: chi sono gli attori, quali sono le risorse, quali sono le minacce plausibili, quali mitigazioni esistono.

Roadmap Migrazione verso OAuth 2.1

OAuth 2.1 non è ancora uno standard definitivo (RFC) al momento della scrittura, ma i principali AS (Keycloak, Auth0, Okta) lo implementano già. La migrazione può essere fatta gradualmente.

Azione migrazione	Impatto	Priorità	Note
Aggiungi PKCE a tutti i client	Basso	Alta	Retrocompatibile con OAuth 2.0
Rimuovi Implicit Flow	Medio	Alta	Sostituisci con Auth Code + PKCE
Rimuovi ROPC	Alto	Alta	Analizza chi lo usa, migra a Auth Code
Exact-match <code>redirect_uri</code>	Basso-Medio	Alta	Verifica tutti gli URI registrati
Rimuovi token in query param	Basso	Media	Controlla tutti i client
Abilita refresh token rotation	Basso	Media	Configura nell'AS

La migrazione più impattante è la rimozione del ROPC grant type, usato spesso in applicazioni legacy dove la gestione del `redirect` era difficile. Prima di rimuoverlo, occorre identificare tutti i client che lo usano (analisi dei log del token endpoint) e pianificare la migrazione uno per uno.

PKCE è la migrazione più semplice: può essere aggiunta senza rompere le applicazioni esistenti, perché un AS configurato correttamente accetta sia richieste con che senza PKCE durante il periodo di transizione, poi rende PKCE obbligatorio con un flag di configurazione.

Il vantaggio principale di completare la migrazione a OAuth 2.1 non è solo la conformità allo standard futuro, ma la riduzione della superficie di attacco: ogni feature rimossa è una potenziale vulnerabilità eliminata. La semplicità è sicurezza.

FAQ e Pattern Avanzati

Domande Frequenti (FAQ)

Q: OAuth 2.0 e autenticazione sono la stessa cosa?

R: No. OAuth 2.0 è un framework di autorizzazione (cosa puoi fare), non di autenticazione (chi sei). Usare OAuth 2.0 per autenticare gli utenti è un anti-pattern comune detto 'Using OAuth for Authentication'. Per l'autenticazione usa OpenID Connect (OIDC), che è costruito sopra OAuth 2.0 e aggiunge l'ID token JWT con informazioni sull'identità dell'utente.

Q: Quanto deve durare un access token?

R: La risposta dipende dal contesto. Per API consumer: 1 ora è lo standard de facto. Per sistemi ad alta sicurezza (finance, sanità, industria critica): 5-15 minuti. Un token con vita più breve riduce la finestra di esposizione in caso di furto, ma aumenta la frequenza dei refresh. Con HTTPS e token storage sicuro, 1 ora è accettabile per la maggior parte dei casi d'uso.

Q: Devo usare PKCE anche con un client confidenziale?

R: Sì, OAuth 2.1 lo rende obbligatorio per tutti i client. Anche per i client confidenziali (con client_secret), PKCE aggiunge uno strato di protezione: se il code venisse intercettato durante il redirect, l'attaccante non potrebbe scambiarlo senza il code_verifier. È retrocompatibile con tutti gli AS moderni.

Q: Posso usare localStorage per memorizzare i token in una SPA?

R: No. localStorage è accessibile da qualsiasi script JavaScript nella pagina, inclusi script di terze parti (analytics, chat widget). Un attacco XSS che inietta JavaScript malevolo può esfiltrare tutti i token. Usa la memoria (variabile JavaScript, non persistente) per gli access token. Per il refresh token, se necessario, usa cookie HttpOnly+Secure (non accessibili da JS).

Q: Come gestisco il 'silent refresh' per evitare che l'utente veda il login?

R: Il silent refresh usa un iframe nascosto per fare il flusso Authorization Code con prompt=none: se l'utente ha ancora una sessione SSO valida, ottiene un nuovo token senza interazione. Se la sessione è scaduta, riceve un errore e deve fare il login completo. Con refresh token in HttpOnly cookie, il silent refresh non è necessario: basta chiamare il token endpoint con il refresh token.

Q: OAuth 2.0 funziona offline (senza connessione Internet)?

R: I JWT già emessi continuano a essere validi offline perché la validazione è crittografica e non richiede round-trip all'AS. Il problema sorge al refresh: senza connessione non si possono ottenere nuovi token. Per sistemi industriali offline, si usano: access token con vita lunga (ore/giorni), token pre-generati per le operazioni offline previste, o un AS locale (Keycloak on-premise nella rete OT).

Q: Come proteggerò un'API pubblica con OAuth senza richiedere login?

R: Per accesso anonimo limitato (rate-limited), puoi emettere token di accesso pubblico senza autenticazione, con scope molto limitati. Tecnicamente: un endpoint /api/public-token emette un token con scope 'public:read', vita breve, rate-limited per IP. Non è OAuth 2.0 standard, ma è un pattern usato

(es. Spotify Web API per ricerche senza login). Per accesso completo, richiedi sempre autenticazione.

Q: Posso implementare il mio Authorization Server?

R: Tecnicamente sì, ma è fortemente sconsigliato. Implementare un AS conforme agli RFC è un progetto complesso: occorre gestire correttamente storage sicuro dei segreti, generazione crittografica di token, validazione rigorosa di tutti i parametri, gestione delle sessioni, revoca, PKCE, OIDC. Un errore in uno di questi punti crea vulnerabilità critiche. Usa sempre un AS maturo e testato (Keycloak, Auth0, Okta, Azure AD) invece di implementarne uno custom.

Q: OAuth 2.0 protegge contro MITM?

R: OAuth 2.0 richiede HTTPS per tutti gli endpoint, ma non garantisce esso stesso la protezione MITM: la delega a TLS. Se TLS è configurato correttamente (certificato valido, protocolli moderni, HSTS abilitato), OAuth 2.0 è protetto da MITM. Se TLS è implementato male (certificato self-signed accettato senza verifica, TLS 1.0 abilitato, HSTS mancante), OAuth 2.0 è vulnerabile come qualsiasi altra comunicazione HTTPS.

Q: Come integro OAuth 2.0 con Active Directory?

R: Microsoft Azure AD (Entra ID) e ADFS supportano OAuth 2.0 e OIDC nativamente e si integrano con Active Directory. Per ambienti on-premise, Keycloak supporta federazione LDAP con AD: gli utenti AD si autenticano con le loro credenziali AD tramite Keycloak, che emette i token OAuth/OIDC. Configurazione in Keycloak: User Federation → LDAP → connessione al Domain Controller AD.

Pattern Architetture Avanzati

Pattern: API Gateway con OAuth Sidecar

Il pattern Sidecar per OAuth sposta la logica di autenticazione fuori dall'applicazione e in un container separato (sidecar) che gira affianco all'applicazione principale nel pod Kubernetes. L'applicazione principale non gestisce direttamente OAuth: tutte le richieste entrano prima nel sidecar, che valida il token, e se valido forwarda la richiesta all'app con i claims JWT negli header HTTP (X-User-Id, X-User-Email, X-User-Roles).

Il sidecar è un servizio HTTP leggero (Envoy, Nginx con lua, o un custom proxy Python/Go) configurato per lo specifico AS e JWKS endpoint. L'applicazione principale può essere scritta in qualsiasi linguaggio senza librerie OAuth: si fida degli header iniettati dal sidecar, che sono impostati solo se il token è valido.

Vantaggi: separazione delle responsabilità, l'applicazione non dipende da librerie OAuth/JWT, configurazione centralizzata, aggiornamento della logica OAuth senza toccare l'app principale. Svantaggi: latenza aggiuntiva (un hop extra), il sidecar è un punto di fallimento critico, richiede Kubernetes o orchestratore simile.

Pattern: BFF — Backend For Frontend

Il pattern BFF (Backend For Frontend) è particolarmente utile per SPA e app mobile. L'idea: invece di gestire OAuth direttamente nel browser/app, si implementa un backend leggero (il BFF) che gestisce tutto il flusso OAuth e mantiene i token in sessioni server-side. Il browser/app comunica solo con il BFF tramite cookie di sessione.

Vantaggi del BFF per OAuth: il client_secret rimane nel backend, i token non sono mai nel browser, il BFF può essere scritto nel linguaggio preferito, il refresh token rotation è trasparente per il frontend, la revoca è semplice (distruggi la sessione server-side).

Il BFF pattern è raccomandato da OAuth 2.0 Security BCP (RFC 9700) per le SPA come alternativa all'uso diretto di refresh token nel browser. Framework come Next.js, Nuxt.js e Angular Universal sono spesso usati come BFF perché hanno già un layer server-side.

Pattern: Step-Up Authentication

Lo step-up authentication richiede che l'utente esegua un'autenticazione aggiuntiva (es. MFA) prima di accedere a operazioni sensibili, anche se ha già una sessione valida. Esempio: l'utente naviga nell'HMI con sessione normale, ma per inviare comandi di controllo al PLC deve confermare con biometrica.

OAuth 2.0 supporta questo tramite il claim ACR (Authentication Context Reference) nell'ID token OIDC. L'AS può essere configurato per richiedere MFA quando il client richiede acr_values=mfa nella richiesta di autorizzazione. Se l'utente ha già una sessione ma senza MFA, l'AS richiede il secondo fattore prima di emettere il token.

■ Step-up authentication con ACR

```
# Step-up authentication con richiesta MFA esplicita
import urllib.parse, requests
```

```

def request_stepup_token(current_token: str, sensitive_resource: str):
    """
    Richiede un token con livello di autenticazione elevato (MFA).
    L'utente verrà reindirizzato per l'MFA se non ha già una sessione MFA.
    """
    params = {
        'response_type': 'code',
        'client_id': CLIENT_ID,
        'redirect_uri': REDIRECT_URI,
        'scope': f'openid {sensitive_resource}:write',
        'state': secrets.token_urlsafe(32),
        'code_challenge': compute_pkce_challenge(),
        'code_challenge_method': 'S256',
        # Step-up: richiede autenticazione con MFA
        'acr_values': 'urn:mace:incommon:iap:silver', # LOA con MFA
        'max_age': '300', # Token MFA non più vecchio di 5 minuti
        'prompt': 'login', # Forza ri-autenticazione se necessario
    }
    # Se l'AS rileva che l'utente non ha MFA nella sessione corrente,
    # mostra la schermata MFA prima di emettere il token
    return f"{AS_AUTHORIZATION_ENDPOINT}?{urllib.parse.urlencode(params)}"

# Verifica ACR nel token ricevuto
def verify_mfa_token(token: str) -> bool:
    payload = jwt.decode(token, PUBLIC_KEY, algorithms=['RS256'],
                        audience=CLIENT_ID, issuer=ISSUER)
    acr = payload.get('acr', '')
    # Verifica che il token includa il claim MFA
    return 'silver' in acr or payload.get('amr', []) in [['mfa'], ['otp'], ['hwk']]

```

Conclusioni e Riepilogo

Abbiamo percorso OAuth 2.0 dalla sua nascita nel 2007 ai pattern più avanzati del 2025. Questo capitolo conclusivo sintetizza i principi fondamentali e fornisce una bussola per le decisioni future.

I 10 Principi Fondamentali di OAuth 2.0

1. OAuth 2.0 è autorizzazione, non autenticazione

Usa OIDC per sapere chi è l'utente. OAuth 2.0 da solo risponde solo alla domanda 'cosa può fare questa app', non 'chi è questo utente'. Confondere i due porta a vulnerabilità gravi.

2. HTTPS è non negoziabile

OAuth 2.0 non ha sicurezza senza TLS. Qualsiasi implementazione che permette HTTP (anche in sviluppo locale) crea abitudini pericolose. Usa certificati self-signed per sviluppo, mai HTTP plain.

3. PKCE per tutti, non solo per i client pubblici

OAuth 2.1 lo rende obbligatorio. PKCE ha zero impatto negativo sulle performance e aggiunge un importante strato di protezione. Implementalo fin dall'inizio.

4. State parameter: sempre, senza eccezioni

Il CSRF contro i flussi OAuth è un attacco reale e documentato. Lo state parameter è la difesa: deve essere casuale, sufficientemente lungo (32 byte), e verificato prima di usare il code.

5. Minimo privilegio negli scope

Chiedere più scope del necessario è pigrizia che diventa vulnerabilità. Un token con scope ampi rubato ha un impatto molto maggiore. Rivedi gli scope di ogni client almeno una volta all'anno.

6. I segreti vanno nei vault, non nel codice

client_secret in Git è un incidente di sicurezza. Usa HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, o simili. Ruota i segreti regolarmente, immediatamente in caso di sospetto.

7. Token brevi + refresh token con rotation

Access token con vita breve limitano l'impatto del furto. Refresh token rotation permette di rilevare uso non autorizzato. Questi due meccanismi insieme sono la difesa più efficace.

8. Revoca al logout, sempre

Fare logout senza revocare i token è una falsa sicurezza. L'utente pensa di essere uscito, ma i token sono ancora validi. Chiama sempre l'endpoint di revoca (RFC 7009) al logout.

9. Valida i JWT come paranoia, non come formalità

Verifica iss, aud, exp, nbf, algoritmo fisso (non dall'header), nonce se OIDC. Una validazione incompleta annulla tutta la sicurezza del sistema OAuth.

10. Monitora, rivedi, migliora

La sicurezza di OAuth 2.0 non è uno stato che si raggiunge, è un processo continuo. Nuove vulnerabilità vengono scoperte, lo standard evolve (OAuth 2.1), le minacce cambiano. Monitora, rivedi periodicamente, e aggiornati.

Il Futuro di OAuth 2.0

OAuth 2.1 consoliderà le best practice degli ultimi 10 anni in un unico documento. PKCE obbligatorio, Implicit rimosso, ROPC rimosso, `redirect_uri exact-match`. Per le nuove implementazioni, seguire già oggi OAuth 2.1 è la scelta corretta.

DPoP (RFC 9449) e mTLS stanno crescendo in importanza per scenari ad alta sicurezza dove i Bearer token (usabili da chi li ruba) non sono sufficienti. DPoP lega il token alla chiave privata del client. mTLS lega il token al certificato client. Entrambi sono più complessi da implementare, ma necessari per scenari come sistemi finanziari critici o infrastrutture industriali esposte a Internet.

I Passkeys (WebAuthn sincronizzati) stanno cambiando l'autenticazione dell'utente finale. L'integrazione con OAuth 2.0/OIDC è già disponibile nei principali AS. Nei prossimi anni, l'autenticazione con password sarà progressivamente sostituita da Passkeys, rendendo MFA trasparente e sicura per gli utenti.

L'EU Digital Identity Wallet (2026) porterà OAuth 2.0 e OIDC al cuore dell'identità digitale europea. Milioni di cittadini useranno wallet conformi OIDC per autenticarsi con servizi pubblici e privati in tutta l'UE. OAuth 2.0 diventerà infrastruttura critica a livello continentale.

Per chi lavora in automazione industriale, la convergenza IT/OT continuerà ad accelerare. PLC, HMI, robot e sistemi SCADA saranno sempre più connessi a infrastrutture cloud. OAuth 2.0 con Client Credentials per la comunicazione M2M, e OIDC per gli operatori umani, diventeranno lo standard de facto anche nel mondo OT.

✓ OAuth 2.0 è oggi il protocollo di autorizzazione dominante sul web e sta diventando lo standard anche nell'IoT industriale. Implementarlo correttamente non è difficile: basta scegliere il grant type giusto, implementare PKCE, gestire i token in modo sicuro, e monitorare il sistema. Buona implementazione!

OAuth 2.0 — Guida Completa

23 capitoli • 10 sezioni di approfondimento • 1 appendice

Appendice B: Esempi di Codice Aggiuntivi

B1 Validazione Token Completa con python-jose

■ Validazione con python-jose

```
# pip install python-jose[cryptography]
from jose import jwt as jose_jwt, JWTError, ExpiredSignatureError
from jose.exceptions import JWEError
import requests, json

ISSUER = 'https://as.example.com'
AUDIENCE = 'my_api'
JWKS_URI = f'{ISSUER}/.well-known/jwks.json'

def get_public_key(token: str):
    """Recupera la chiave pubblica corretta dall'endpoint JWKS."""
    header = jose_jwt.get_unverified_header(token)
    kid = header.get('kid')
    jwks = requests.get(JWKS_URI).json()
    for key in jwks.get('keys', []):
        if key.get('kid') == kid:
            return key
    raise ValueError(f"Chiave non trovata per kid={kid}")

def validate_token(token: str) -> dict:
    key = get_public_key(token)
    try:
        payload = jose_jwt.decode(
            token, key,
            algorithms=['RS256', 'ES256'],
            audience=AUDIENCE,
            issuer=ISSUER,
            options={'verify_exp': True, 'verify_iss': True,
                    'verify_aud': True, 'require_iat': True})
        return payload
    except ExpiredSignatureError:
        raise ValueError("Token scaduto")
    except JWTError as e:
        raise ValueError(f"Token non valido: {e}")

# Test
try:
    claims = validate_token(token_from_request)
    print(f"Utente: {claims['sub']}, Scope: {claims.get('scope')}")
except ValueError as e:
    print(f"Accesso negato: {e}")
```

B2 Client OAuth Asincrono con httpx

■ Client OAuth asincrono con httpx

```
# pip install httpx
import httpx, time, asyncio

class AsyncOAuthClient:
    """Client OAuth 2.0 asincrono per applicazioni async Python (FastAPI, aiohttp)."""
```

```

def __init__(self, token_url, client_id, client_secret, scope):
    self.token_url = token_url
    self.client_id = client_id
    self.client_secret = client_secret
    self.scope = scope
    self._token = None
    self._expiry = 0

    async def get_token(self) -> str:
        if self._token and time.time() < self._expiry - 30:
            return self._token
        async with httpx.AsyncClient() as client:
            resp = await client.post(self.token_url,
                                    data={'grant_type': 'client_credentials', 'scope': self.scope},
                                    auth=(self.client_id, self.client_secret), timeout=10.0)
            resp.raise_for_status()
            d = resp.json()
            self._token = d['access_token']
            self._expiry = time.time() + d.get('expires_in', 3600)
            return self._token

    async def get(self, url: str, **kwargs):
        token = await self.get_token()
        async with httpx.AsyncClient() as client:
            return await client.get(url,
                                    headers={'Authorization': f'Bearer {token}'}, **kwargs)

    async def post(self, url: str, **kwargs):
        token = await self.get_token()
        async with httpx.AsyncClient() as client:
            return await client.post(url,
                                    headers={'Authorization': f'Bearer {token}'}, **kwargs)

# Utilizzo con FastAPI
from fastapi import FastAPI
app = FastAPI()
oauth = AsyncOAuthClient('https://as.example.com/token',
                        'client_id', 'secret', 'api:read')

@app.get('/proxy/data')
async def proxy_data():
    resp = await oauth.get('https://api.example.com/data')
    return resp.json()

# Test standalone
async def main():
    resp = await oauth.get('https://api.example.com/data')
    print(resp.json())

asyncio.run(main())

```

B3 Token Binding con Fingerprinting IP

In alcuni scenari è utile legare la sessione all'IP del client per rilevare potenziali token theft. Non è un meccanismo OAuth standard ma un layer applicativo aggiuntivo adatto per sistemi industriali dove gli operatori hanno IP fissi.

■ IP binding sessione per sistemi industriali

```
import hashlib, time, hmac, os

class IPBoundSessionManager:
    """
    Associa la sessione OAuth all'IP del client.
    Detecta accessi da IP diversi durante la stessa sessione.
    Non uno standard OAuth – layer applicativo per sistemi industriali.
    """
    def __init__(self, secret_key: bytes):
        self.secret = secret_key

    def bind(self, session_id: str, client_ip: str) -> str:
        """Crea un fingerprint IP-bound per la sessione."""
        data = f"{session_id}:{client_ip}:{int(time.time()) // 3600}".encode()
        return hmac.new(self.secret, data, hashlib.sha256).hexdigest()[:16]

    def verify(self, session_id: str, client_ip: str, stored_fp: str) -> bool:
        expected = self.bind(session_id, client_ip)
        return hmac.compare_digest(expected, stored_fp)

# Nel middleware Flask/FastAPI:
# ip_mgr = IPBoundSessionManager(os.environ['SESSION_SECRET'].encode())
#
# Al login:
# session['ip_fp'] = ip_mgr.bind(session.sid, request.remote_addr)
#
# Ad ogni richiesta:
# if not ip_mgr.verify(session.sid, request.remote_addr, session['ip_fp']):
#     log_oauth_event('IP_CHANGE_DETECTED', session.get('user'), ...)
#     session.clear() # Invalida sessione
#     return redirect('/login')
```

Appendice C: Configurazioni di Riferimento

C1 Keycloak realm.json — Export di Configurazione

Keycloak permette di esportare e importare la configurazione di un realm in formato JSON. Utile per replicare la configurazione tra ambienti (es. da staging a produzione) o per infrastructure-as-code.

■ Keycloak realm.json — configurazione impianto acque

```
{
  "realm": "impianto-acque",
  "displayName": "Impianto Trattamento Acque",
  "enabled": true,
  "sslRequired": "external",
  "accessTokenLifespan": 600,
  "refreshTokenMaxReuse": 0,
  "revokeRefreshToken": true,
  "ssoSessionIdleTimeout": 1800,
  "ssoSessionMaxLifespan": 36000,
  "clients": [
    {
      "clientId": "hmi_dashboard",
      "enabled": true,
      "protocol": "openid-connect",
      "publicClient": false,
      "standardFlowEnabled": true,
      "serviceAccountsEnabled": false,
      "redirectUris": ["https://hmi.factory.local/*"],
      "webOrigins": ["https://hmi.factory.local"],
      "defaultClientScopes": ["openid", "profile", "email"],
      "optionalClientScopes": ["roles"]
    },
    {
      "clientId": "scada_system",
      "enabled": true,
      "protocol": "openid-connect",
      "publicClient": false,
      "standardFlowEnabled": false,
      "serviceAccountsEnabled": true,
      "attributes": {
        "access.token.lifespan": "300"
      }
    }
  ],
  "roles": {
    "realm": [
      {"name": "operatore"},
      {"name": "supervisore"},
      {"name": "admin_impianto"}
    ]
  },
  "passwordPolicy": "length(12) and specialChars(1) and upperCase(1) and digits(1) and notRecentlyUsed(5)",
  "browserSecurityHeaders": {
    "contentSecurityPolicy": "frame-src 'self'; frame-ancestors 'self'; object-src 'none';",
    "strictTransportSecurity": "max-age=31536000; includeSubDomains",
    "xFrameOptions": "SAMEORIGIN",
    "xContentTypeOptions": "nosniff"
  }
}
```



```
}  
}
```

C2 .env e docker-compose per sviluppo locale

■ Configurazione docker-compose sviluppo OAuth

```
# .env.example — MAI committare .env con valori reali  
KEYCLOAK_URL=http://localhost:8080  
KEYCLOAK_REALM=dev  
KEYCLOAK_CLIENT_ID=my_app  
KEYCLOAK_CLIENT_SECRET=REPLACE_ME  
API_BASE_URL=http://localhost:8000  
SESSION_SECRET=REPLACE_ME_WITH_RANDOM_32_BYTES  
  
# docker-compose.dev.yml  
version: '3.8'  
services:  
  keycloak:  
    image: quay.io/keycloak/keycloak:latest  
    ports: ['8080:8080']  
    environment:  
      KEYCLOAK_ADMIN: admin  
      KEYCLOAK_ADMIN_PASSWORD: admin_dev  
    command: start-dev --import-realm  
    volumes:  
      - ./keycloak/realm-dev.json:/opt/keycloak/data/import/realm.json  
  
  api:  
    build: ./api  
    ports: ['8000:8000']  
    environment:  
      - KEYCLOAK_URL=http://keycloak:8080  
      - KEYCLOAK_REALM=dev  
      - KEYCLOAK_CLIENT_ID=my_app  
      - KEYCLOAK_CLIENT_SECRET=dev_secret  
    depends_on: [keycloak]  
  
  frontend:  
    build: ./frontend  
    ports: ['3000:3000']  
    environment:  
      - VITE_API_URL=http://localhost:8000  
      - VITE_KC_URL=http://localhost:8080
```

Una buona configurazione di sviluppo locale permette di testare l'intero flusso OAuth senza dipendenze esterne. Keycloak in modalità start-dev è adatto per il locale: usa un database H2 in-memory, avvia rapidamente, permette di importare un realm pre-configurato all'avvio.

Il file .env.example serve come documentazione delle variabili d'ambiente necessarie senza esporre i valori reali. Ogni sviluppatore copia .env.example in .env e imposta i propri valori. Il file .env non deve mai essere committato: aggiungilo al .gitignore fin dalla prima riga di codice del progetto.

Appendice: Riferimento Rapido e Glossario

A1 RFC di Riferimento

RFC	Titolo	Contenuto
RFC 6749	OAuth 2.0 Authorization Framework	Core OAuth 2.0
RFC 6750	Bearer Token Usage	Come usare i Bearer token
RFC 7009	Token Revocation	Endpoint di revoca
RFC 7519	JSON Web Token (JWT)	Formato JWT
RFC 7636	PKCE	PKCE per client pubblici
RFC 7662	Token Introspection	Verifica token attivi
RFC 8252	OAuth for Native Apps	Best practice app mobile
RFC 8628	Device Authorization Grant	Device Flow
RFC 8693	Token Exchange	Scambio token
RFC 9126	Pushed Authorization Requests	PAR
OpenID Core	OpenID Connect Core 1.0	Specifica OIDC
OAuth 2.1	OAuth 2.1 (draft)	Versione consolidata futura

A2 Quale Grant Type Scegliere?

Scenario	Grant Type	Note
Web app con backend	Authorization Code + PKCE	Standard OAuth 2.1
SPA (Single Page App)	Authorization Code + PKCE	PKCE senza client_secret
App mobile iOS/Android	Authorization Code + PKCE	Universal Links o custom scheme
CLI tool	Device Flow o Auth Code+PKCE	Device se no browser
M2M / microservizi	Client Credentials	Nessun utente coinvolto
PLC / IoT	Client Credentials	Edge proxy se risorse limitate
Smart TV / console	Device Flow	Utente usa un secondo device
Login sociale (SSO)	Auth Code + PKCE + OIDC	Aggiunge ID token

A3 Glossario

Access Token: Credenziale temporanea per accedere alle API protette (vita breve, ~1h).

Authorization Code: Codice monouso (10 min) scambiato per token nel backend.

Authorization Server: Server che autentica l'utente ed emette i token.

Bearer Token: Token che funziona per chiunque lo presenti, senza prova di identità.

Claim: Coppia chiave-valore nel payload JWT (sub, email, exp, ...).

Client: App che richiede accesso alle risorse per conto dell'utente.

Client Credentials: client_id e client_secret dell'app registrata.

Code Challenge: Hash SHA-256 del code_verifier (PKCE), inviato nell'URL.

Code Verifier: Segreto casuale generato dal client prima del flusso PKCE.

Discovery: /.well-known/openid-configuration — descrive automaticamente l'AS.

Grant Type: Meccanismo per ottenere un token (auth_code, client_credentials, ...).

ID Token: JWT OIDC con informazioni sull'utente autenticato.

JWKS: JSON Web Key Set — chiavi pubbliche dell'AS per verificare JWT.

JWT: JSON Web Token — formato self-contained, firmato crittograficamente.

Nonce: Valore casuale incluso nella richiesta OIDC per prevenire replay.

OIDC: OpenID Connect — strato di autenticazione sopra OAuth 2.0.

PKCE: Proof Key for Code Exchange — sicurezza per client pubblici (RFC 7636).

Refresh Token: Token a lunga vita per rinnovare l'access token senza ri-autenticazione.

Resource Owner: L'entità (utente) che possiede le risorse protette.

Resource Server: Server che ospita le risorse protette, accetta access token.

Revocation: Invalidazione del token prima della scadenza naturale (RFC 7009).

Scope: Stringa che definisce i permessi richiesti (email, profile, repo, ...).

State: Valore casuale per protezione CSRF nel Authorization Code Flow.

A4 OAuth 2.1 — Le Novità Principali

Cambiamento	OAuth 2.0	OAuth 2.1
PKCE	Opzionale	Obbligatorio per tutti
Implicit Flow	Definito	Rimosso
ROPC	Definito	Rimosso
redirect_uri validation	Match parziale OK	Match esatto richiesto
Bearer token in query param	Permesso (deprecated)	Vietato
Refresh token rotation	Opzionale	Fortemente raccomandata

Buona implementazione! ■